

# Efficient Single-Round Obfuscation of Search and Result Patterns in Searchable Encryption

Tung Le  
Virginia Tech  
Blacksburg, VA, USA  
tungle@vt.edu

Thang Hoang  
Virginia Tech  
Blacksburg, VA, USA  
thanghoang@vt.edu

## Abstract

Searchable Symmetric Encryption (SSE) enables data owners to securely store encrypted data on untrusted cloud servers while retaining the ability to perform secure searches and retrieve relevant documents. However, standard SSE schemes expose search patterns (whether two queries are identical), and result patterns (which documents are returned), making them susceptible to leakage-abuse attacks that can infer sensitive information such as the queried keywords and/or document contents. While Oblivious RAM (ORAM) and Private Information Retrieval (PIR) can hide these patterns, their high computation and communication overhead often render them impractical for real-world search workloads. A more efficient alternative is to obfuscate search and result patterns using Differential Privacy (DP). Unfortunately, existing DP-based SSE schemes either provide insufficient query privacy protection, or still incur substantial performance overhead.

In this paper, we propose FROST, a novel differentially private SSE scheme that efficiently obfuscates both search and result patterns, while providing strong resilience against all known statistical leakage-abuse attacks. The core component of FROST is our new rerandomized PIR (RePIR) scheme designed for private databases, which allows server-side rerandomization of encrypted PIR query responses. In FROST, we also introduce a novel method for applying DP noises to SSE for search result obfuscation using only simple arithmetic operations. An important property of FROST is that it requires only a single round of communication, with small user-side storage as an additional benefit. We fully implemented FROST and conducted extensive experiments over real-world datasets to rigorously assess its practical performance and resilience. Our experiments showed that FROST not only effectively mitigates pattern-leakage attacks while maintaining reasonable utility, but also achieves up to *four orders of magnitude faster* keyword search and *three orders of magnitude lower bandwidth* overhead than prior DP-based SSE schemes.

## CCS Concepts

• Security and privacy → Database and storage security; Privacy-preserving protocols; Cryptography.

## Keywords

Searchable Encryption, Access Pattern, Differential Privacy



This work is licensed under a Creative Commons Attribution 4.0 International License.  
CCS '26, The Hague, Netherlands

© 2026 Copyright held by the owner/author(s).  
ACM ISBN 979-8-4007-2871-6/2026/11  
<https://doi.org/10.1145/3830454.3832727>

## ACM Reference Format:

Tung Le and Thang Hoang. 2026. Efficient Single-Round Obfuscation of Search and Result Patterns in Searchable Encryption. In *Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security (CCS '26)*, November 15–19, 2026, The Hague, Netherlands. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3830454.3832727>

## 1 Introduction

Searchable Symmetric Encryption (SSE) [81] enables a user to conduct secure searches over encrypted data stored remotely on a server. Typically, the user encrypts both the database and an associated search index on their local machine before outsourcing them to the server. When a search is requested, the user generates a query that allows the server to perform a secure lookup on the encrypted index and return the relevant encrypted documents, which the user can then decrypt on their end.

Although both queries and documents are encrypted, the server can still infer sensitive information during the user's interaction. In particular, it can observe which documents are retrieved (revealing the result patterns) and identify repeated queries (revealing the search patterns). Many existing SSE schemes [12, 16, 25, 52, 53, 64, 81] permit this type of leakage to maintain efficiency for keyword search. However, a lot of research [14, 43, 50, 73, 75, 79, 97] has shown that if the server has some background or partial similar knowledge about a portion of the stored data or queries, it can infer the actual queried keywords and/or document content with high accuracy, posing a severe threat to user and data privacy.

Many improvements have been developed to strengthen the privacy guarantees of SSE systems, but these often come with trade-offs in performance. These trade-offs typically include increased communication costs, higher computational demands, and additional storage burdens on the user side. Solutions using Oblivious RAM (ORAM) [41, 82, 88] or Private Information Retrieval (PIR) [22, 34, 40, 45, 61] can completely conceal which documents are accessed in a database. However, ORAM introduces significant communication overhead, while PIR incurs substantial computation cost, and both are generally not tailored for secure keyword search in encrypted databases (an exception for TWORAM [37]). Another line of research uses distributed/multiparty computation (MPC) with multiple servers [27, 33, 62, 63] to hide search and result patterns, relying on the assumption that at least one server is honest (i.e., the servers do not all collude). To actually retrieve query-matching documents, these protocols often necessitate multiple communication rounds, which is an important criterion of searchable encryption [25] and should be kept minimal. Connectivity is relatively scarce in isolated areas and mobile IoT networks,

**Table 1: Comparison of FROST with prior encrypted search systems.**

| Scheme                     | #Servers<br>$L$ | Search Leakage<br>$\mathcal{L}_{\text{Search}}$                        | Forward Privacy | #Rounds | Search Complexity         |                 |                         |
|----------------------------|-----------------|------------------------------------------------------------------------|-----------------|---------|---------------------------|-----------------|-------------------------|
|                            |                 |                                                                        |                 |         | Server                    | User            | Communication           |
| SSE [15]                   | 1               | $\{\text{sp}(\text{kw}), \text{rp}(\text{kw}), \text{sv}(\text{kw})\}$ | ✗               | 1       | $O(a_w)$                  | $O(\lambda)$    | $O(\lambda + n_w)$      |
| $\Sigma\phi\phi\phi$ [12]  | 1               | $\{\text{sp}(\text{kw}), \text{rp}(\text{kw}), \text{sv}(\text{kw})\}$ | ✓               | 1       | $O(a_w)$                  | $O(\lambda)$    | $O(\lambda + n_w)$      |
| CLRZ [19]                  | 1               | $\{\text{sp}(\text{kw}), \text{sv}(\text{kw})\}$                       | ✗               | 1       | $O(a_w)$                  | $O(\lambda)$    | $O(\lambda + n_w)$      |
| OSSE [79]                  | 1               | $\{\emptyset\}$                                                        | ✗               | 1       | $O(d_w N^\dagger)$        | $O(d_w N)$      | $O(d_w N)$              |
| DORY [27]                  | 2               | $\{\emptyset\}$                                                        | ✓               | 2*      | $O(mN)$                   | $O(\log m + N)$ | $O(\lambda \log m + N)$ |
| Branch [92]                | 1               | $\{\text{sp}(\text{kw}), \text{rp}(\text{kw}), \text{sv}(\text{kw})\}$ | ✓               | 1       | $O(a_w)$                  | $O(\lambda)$    | $O(\lambda + n_w)$      |
| L-SD <sub>d</sub> [1C][71] | 1               | $\{\text{sp}(\text{kw}), \text{rp}(\text{kw}), \text{sv}(\text{kw})\}$ | ✓               | 1       | $O(\log N_{\text{kw},f})$ | $O(\lambda)$    | $O(\lambda + n_w)$      |
| Our FROST                  | 1               | $\{\emptyset\}$                                                        | ✓               | 1       | $O(mN)$                   | $O(m + N)$      | $O(m + N)$              |

•  $\text{kw}$ : the input search keyword;  $\text{sp}$ : Search pattern;  $\text{rp}$ : Result pattern;  $\text{sv}$ : Search volume (see §2.2 for more details).

•  $\lambda$ : Security parameter;  $N$ : Total number of documents;  $d_w$ : Number of keywords per document;  $N_{\text{kw},f}$ : Number of (keyword, file) pairs;  $m$ : Size of keyword representation *per* document;  $a_w$ : Number of times the queried keyword  $w$  is historically added (updated) to the database;  $n_w$ : Number of matched results for keyword  $w$ .

• In practice,  $d_w < m \ll W$ , where  $W$  is the keyword universe set, and  $n_w \leq N$ .

We assume document identifiers can be represented using a constant number of bits to skip this quantity in search complexity.

† Public-key pairing operations.

\* DORY [27] requires two communication rounds to search and retrieve documents. In DORY, the user needs to decrypt search results computed by the server first before being able to retrieve documents in the next round.

and round-trip latency can be large. Therefore, minimizing user-server interactions (preferably one round) is often a major goal (e.g., [79]).

To address result pattern leakage in SSE with a single round-trip, a differential privacy (DP)-based approach by Chen et al. [19] involves obfuscating the database index prior to outsourcing it. This ensures the server only observes obfuscated result patterns, thereby limiting the potential for attacks by such leakage. Nevertheless, while this method improves security, it does not conceal search patterns, since the result pattern for each keyword is determined and remains unchanged once the database is outsourced. As a result, search pattern leakage still enables several real-world attacks [50, 67, 73–76] to be effective, even when result patterns are obfuscated. The follow-up work OSSE by Shang et al. [79] obfuscates both search and result patterns but incurs substantial computational and bandwidth overhead, which takes about 27 minutes to process a search query over only 30K documents on a 160-core powerful server [79, Section IX].

Regarding the security of OSSE, applying DP at the document-level granularity, as OSSE does, is still an effective approach to mitigate existing attacks. The most recent attack on querying schemes that use DP to protect query and data privacy focuses on “bucketization” and “partitions with encodings” techniques, which produce noise at the bucket or partition level during user interactions when executing search [77]. However, these schemes always return the requested bucket or partition in the result. The attack leverages the fact that buckets or partitions returned as result in the same set of queries would correspond to the same label. Consequently, by observing the relation in results of sufficient queries, an adversary can estimate the bounds on label values of buckets or partitions, then recover the histogram of labels in data. Given that OSSE remains secure against state-of-the-art attacks under proper differential-privacy parameters, our goal is to improve its performance while preserving its security guarantees.

In this work, we propose FROST, which stands for Searchable Encryption with Efficient Single-Round Obfuscation of Search and Result Patterns, a new SSE scheme that protects both search and result patterns with high performance and only requires a *single-round* communication. The main idea behind FROST is that it produces a fresh obfuscation per query by adding false positives and false negatives to the output results through lightweight arithmetic operations. More specifically, the high-level idea of FROST search is to add a random mask onto PIR output, which is known by the user, followed by using dot product operator with transformation vectors to set the output as true positives, false positives, or random non-matches. Our approach is fundamentally different from OSSE, which requires a large number of additional tokens to obfuscate real queries, as well as matches and unmatched results, incurring high computation and bandwidth costs. Instead, we design RePIR, a new scheme for private information retrieval on a private database, which outputs the queried data masked with random values that can be precomputed, facilitating obfuscation for the search results later. Next, we propose a novel approach for applying DP in SSE. Intuitively, existing works that hide pattern leakage [27, 48, 62] structure the search index as a table, where the value ‘1’ at row  $f$  and column  $w$  indicates that keyword  $w$  appears in document  $f$ , and ‘0’ otherwise. Executing a keyword search is privately retrieving the column corresponding to the queried keyword and filter out matched documents with value ‘1’. Several works apply Bloom filter [27, 33, 62, 63] to compress the search index, at the cost of checking more columns in the online phase. We instead represent these states ‘0’ and ‘1’ as integer vectors, which allows us to leverage dot products to transform these state vectors into arbitrary scalar values. For keyword search, the user sends transformation vectors specifically constructed to compute dot products with the state vectors, thereby collapsing them into precomputable scalar values. Therefore, the user can configure the transformation vectors to classify search results to true positives, false positives, or random non-matches. Our approach with RePIR and state/transformation vectors achieves

significantly better performance by only using lightweight operations (i.e., additions/multiplications between 32 and 64-bit integers), as opposed to OSSE, which relies on computation-heavy cryptographic techniques (i.e., public-key pairing). Also, FROST allows to perform queries on the encrypted database and receive the matched documents in the same communication round. (MUSES [62] and TWORAM [37] require at least 3 and 4 rounds, respectively). Moreover, it requires small user-side storage. Our experimental evaluation showed that applying DP is highly effective to mitigate state-of-the-art query recovery attacks for SSE while maintaining efficiency for keyword search.

Briefly, inspired by the prior work OSSE [79], which establishes a strong formal foundation for DP-based SE, our FROST targets the same DP objective while significantly improving performance. In summary, our contributions are the following:

- We propose FROST, an SSE scheme whose main features include: (i) obfuscation of both search and result patterns; (ii) single-round communication with small user-side storage; and (iii) high resilience against known practical query-recovery attacks, which assume that adversaries have access to a substantial amount of similar data as auxiliary background information, as much as half of the target user's dataset (see details in §6).
- We introduce RePIR, a novel primitive for efficient private information retrieval on private databases that supports rerandomization on the PIR encrypted output, setting the stage for subsequent secure computations on the retrieved masked data.
- We introduce a new approach to applying DP in SSE effectively. Our method builds the search index using state vectors, which are rerandomized upon retrieval through RePIR. We then apply transformation vectors to output obfuscated search results.
- We fully implemented FROST and evaluated its performance on commodity servers. Under real environments, experimental results demonstrated that FROST performs search  $460\times$ – $10012\times$  faster with  $1152\times$ – $3748\times$  smaller bandwidth overhead than OSSE [79] with the same security guarantees. FROST renders leakage-abuse attacks leveraging search-pattern and query-frequency leakage [67, 73] largely ineffective, reducing attack accuracy from 39.9–81.4% to near-random guessing ( $\approx 0.1\%$ ) (see §6.1.1). Our implementation is available at <https://github.com/vt-asaplab/FROST>.

Table 1 compares FROST with prior SSE designs. To our knowledge, FROST is the first SSE that can obfuscate search result patterns and achieve efficiency simultaneously without using costly oblivious access primitives or distributed servers. Furthermore, FROST achieves single-round search, which is important in practice for high-latency or intermittently connected settings, as it allows the user to issue a query and later obtain the result without sustained interaction. We stress that achieving single-round search with pattern obfuscation like FROST is technically challenging, since the server must directly output the final search result in a single response, with no room for extra user-side post-processing (e.g., noise addition). By contrast, multi-round protocols can leverage existing techniques such as ORAM or PIR to decouple query execution from result obfuscation, at the cost of additional interaction.

## 2 Preliminaries

**Notation.**  $\parallel$  denotes the concatenation operator. We denote by  $\lambda$  the security parameter and by  $\mathbb{Z}_p$  the ring of integer scalars modulo  $p$ . We denote  $\hat{\mathbb{Z}}_p$  the ring of integer vectors modulo  $p$ . We denote by  $[n]$  the set  $\{1, \dots, n\}$ .  $x \xleftarrow{\$} [n]$  means  $x$  is selected uniformly at random from the set  $\{1, \dots, n\}$ . Bold small letters (**a**) denote vectors, while capitalized bold letters (**M**) denote matrices. We denote by  $\langle \mathbf{a}, \mathbf{b} \rangle$  the dot product of **a** and **b**. Given a vector **a**, we denote  $|\mathbf{a}|$  as the cardinality of vector **a**, and  $\mathbf{a}[i]$  as the  $i$ -th element of vector **a**. Given a matrix **M**,  $\mathbf{M}[i, *]$  and  $\mathbf{M}[:, j]$  denote accessing the row  $i$  and column  $j$  of **M**, respectively.  $\mathbf{M}[i, j]$  denotes accessing the cell indexed at row  $i$  and column  $j$ . We denote  $A \stackrel{c}{\approx} B$  as the computational indistinguishability between two distributions  $A$  and  $B$ .

Let  $W = \{w_1, w_2, \dots, w_{|W|}\}$  be the keyword universe, and let  $|W|$  denote its size. Let  $\mathcal{D}$  be a dataset of  $N$  documents, sorted by their ids.  $\mathcal{D}[i]$  is the  $i$ -th document in the dataset; without loss of generality, we assume  $\text{id}(\mathcal{D}[i]) = i$ . Let  $\mathcal{D}(w)$  be the list of documents that contain  $w$  ordered by id.

Let  $\text{BF} = (\text{Init}, \text{Gen}, \text{Vrfy})$  be a Bloom filter (BF) [10]:  $(H_1, \dots, H_K) \leftarrow \text{BF.Init}(m, K)$ : generating  $K$  mappings  $H_k : \mathcal{S} \rightarrow [m] \forall k \in [K]$  with two parameters  $m$  (BF size) and  $K$ ;  $\mathbf{u} \leftarrow \text{BF.Gen}(\mathcal{S})$ : computing the BF representation  $\mathbf{u} \in \{0, 1\}^m$  of a given set  $\mathcal{S}$ ;  $\{0, 1\} \leftarrow \text{BF.Vrfy}(\mathbf{u}, s)$ : checking if an element  $s$  belongs to the set represented by  $\mathbf{u}$ . We denote by  $\hat{d} \leftarrow \lceil d \rceil_{\Delta} / \Delta$  rounding  $d$  to the nearest multiple of  $\Delta$  then divided by  $\Delta$ , with  $\Delta \in \mathbb{Z}$ , and by  $\tilde{d} \leftarrow \lfloor d \rfloor_{\Delta} / \Delta$  rounding down  $d$  to the nearest multiple of  $\Delta$  then divided by  $\Delta$ .

**Bit Operations.** We denote  $x \ll t$  and  $x \gg t$  as the (binary) left-shift and right-shift operations by  $t$  bits of value  $x$ .

### 2.1 Learning With Errors (LWE)

The security of our RePIR scheme (see §4.2) relies on the decision version of the learning-with-errors assumption [78].

**Definition 1 (LWE).** Given a set of  $n$  basis vectors  $\mathbf{a}_i \in \mathbb{Z}_q^m$  represented by matrix **A** and a point  $\mathbf{t} \in \mathbb{Z}_q^m$ . The LWE is the problem of determining a set of secret coefficients  $\mathbf{s} = (s_1, s_2, \dots, s_n)$  with  $s_i \in \mathbb{Z}_q$ , such that  $\mathbf{A} \cdot \mathbf{s} + \mathbf{e} = \mathbf{t} \bmod q$ , where  $\mathbf{e}$  is an unknown error vector consisting of small integers modulo  $q$ .

The assumption is parameterized by the dimension of the LWE secret  $n \in \mathbb{N}$ , the number of samples  $m \in \mathbb{N}$ , the integer modulus  $q \geq 2$ , and an error distribution  $\chi$  over  $\mathbb{Z}$ . The LWE assumption then asserts that for a matrix  $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{m \times n}$ , a secret  $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^n$ , an error vector  $\mathbf{e} \xleftarrow{\$} \chi^m$ , and a random vector  $\mathbf{r} \xleftarrow{\$} \mathbb{Z}_q^m$ , the following distributions are computationally indistinguishable:

$$\{(\mathbf{A}, \mathbf{A} \cdot \mathbf{s} + \mathbf{e})\} \stackrel{c}{\approx} \{(\mathbf{A}, \mathbf{r})\}.$$

More specifically, the  $(n, q, \chi)$ -LWE problem with  $m$  samples is  $(T, \tilde{\epsilon})$ -hard if all adversaries running in time  $T$  have advantage at most  $\tilde{\epsilon}$  in distinguishing between the two distributions.

**Secret-key Regev Encryption.** Regev [78] gives a secret-key encryption scheme that is secure under the LWE assumption. With LWE parameters  $(n, q, \chi)$  and a plaintext modulo  $p$ , the Regev secret

key is a vector  $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^n$ . The Regev encryption of  $\mu \in \mathbb{Z}_p$  is:

$$(\mathbf{a}, c) = (\mathbf{a}, \langle \mathbf{a}, \mathbf{s} \rangle + e + \lfloor q/p \rfloor \cdot \mu) \in \mathbb{Z}_q^n \times \mathbb{Z}_q,$$

for  $e \xleftarrow{\$} \chi$ . To decrypt the ciphertext, anyone who knows the secret  $\mathbf{s}$  can compute  $(c - \langle \mathbf{a}, \mathbf{s} \rangle) \bmod q$  and round the result to the nearest multiple of  $\lfloor q/p \rfloor$ . Decryption succeeds as long as the absolute value of the error sampled from the error distribution  $\chi$  is smaller than  $\frac{1}{2} \cdot \lfloor q/p \rfloor$ . We say that a setting of the Regev parameters supports *correctness error*  $\delta$  if the probability of a decryption error is at most  $\delta$  (over the encryption algorithm's randomness).

## 2.2 Searchable Encryption

Searchable Encryption permits a user to perform privacy-preserving keyword searches over encrypted documents without leaking the plaintext of the keyword and documents to the storage server. We recall Dynamic Searchable Symmetric Encryption (DSSE) that allows both secure keyword search and document update.

**Definition 2 (DSSE).** A DSSE scheme is a tuple of PPT algorithms defined as follows:

- $(\kappa, \text{st}, \text{EDB}) \leftarrow \text{Setup}(1^\lambda)$ : Given a security parameter  $\lambda$ , it outputs a secret key  $\kappa$  and a state  $\text{st}$  to be stored locally, and an initialized encrypted database EDB.
- $\mathbf{s} \leftarrow \text{SrchTkn}(\text{st}, \kappa, \mathbf{w})$ : Given a state  $\text{st}$ , a secret key  $\kappa$ , and a keyword  $\mathbf{w}$ , it outputs a search token  $\mathbf{s}$ .
- $\mathcal{R} \leftarrow \text{Srch}(\mathbf{s}, \text{EDB})$ : Given a search token  $\mathbf{s}$  and an encrypted database EDB, it outputs the search result  $\mathcal{R}$ .
- $(\mathbf{u}, \text{st}') \leftarrow \text{UpdtTkn}(\text{st}, \kappa, f, \mathcal{W}_f)$ : Given a state  $\text{st}$ , a secret key  $\kappa$ , a document  $f$  and its keyword set  $\mathcal{W}_f$ , it outputs an update token  $\mathbf{u}$  and a new state  $\text{st}'$ .
- $\text{EDB}' \leftarrow \text{Updt}(\mathbf{u}, \text{EDB})$ : Given an update token  $\mathbf{u}$  and an encrypted database EDB, it outputs an updated encrypted database  $\text{EDB}'$ .

**Definition 3 (Correctness of DSSE).** For all security parameter  $\lambda$ , all  $(\kappa, \text{st}, \text{EDB}) \leftarrow \text{Setup}(1^\lambda)$ , and all sequences of  $\text{Srch}$ ,  $\text{Updt}$  operations over encrypted database EDB using tokens generated respectively from  $\text{SrchTkn}(\text{st}, \kappa, \mathbf{w})$ , and  $\text{UpdtTkn}(\text{st}, \kappa, f, \mathcal{W}_f)$ ,  $\text{Srch}$  returns the correct results w.r.t the inputs  $(f, \mathcal{W}_f)$  of  $\text{UpdtTkn}$ , except with negligible probability in  $\lambda$ .

To define the security of a DSSE scheme, we borrow the concepts of *history*, *view* and *trace* [79]. The history contains the sensitive information that the user wants to keep private, namely the document plaintexts  $\mathcal{D}$  and the sequence of queried keywords  $\mathbf{w}$ :

**Definition 4 (History).** A history over  $\mathcal{D}$  is a tuple:

$$H_t = (\mathcal{D}, \mathbf{w}), \quad \text{where } \mathbf{w} = (\mathbf{w}[1], \mathbf{w}[2], \dots, \mathbf{w}[t]).$$

Here,  $\mathbf{w}$  is the vector of underlying keywords of the  $t$  queries.

A *partial history* of  $H_t$ , denoted  $H_t^s$  for  $s \leq t$ , contains only the sequence of queries up to the  $s$ -th query, i.e.,  $H_t^s = (\mathcal{D}, \mathbf{w}')$ , where  $\mathbf{w}' = (\mathbf{w}[1], \dots, \mathbf{w}[s])$ .

Next, the view specifies what the server can see when running the DSSE protocol, namely the document identifiers  $\text{id}(\mathcal{D})$ , the encryption of each document  $\mathcal{E}(\mathcal{D}[i])$ , an encrypted database EDB, and the query tokens (trapdoors)  $\mathbf{s}_i$ :

**Definition 5 (View).** The view of  $H_t$  under key  $\kappa$  is the vector:

$$V_\kappa(H_t) = (\text{id}(\mathcal{D}), \mathcal{E}(\mathcal{D}[1]), \dots, \mathcal{E}(\mathcal{D}[N]), \text{EDB}, \mathbf{s}_1, \dots, \mathbf{s}_t, \mathbf{u}_1, \dots, \mathbf{u}_t)$$

where  $\mathbf{s}_i$  and  $\mathbf{u}_i$  are the search query token and the update token for the  $i$ -th query, respectively. The partial view  $V_\kappa^s(H_t)$  only contains the tokens up to  $\mathbf{s}_s$  and  $\mathbf{u}_s$ , with  $s \leq t$ .

Finally, the *trace* of a history models the actual leakage of the DSSE scheme. Before defining trace, we first formulate three types of leakage of DSSE schemes, including the search pattern, result pattern, and search volume as follows.

**Definition 6 (Search Pattern).** The search pattern (sp), denoted as  $\Phi_{\mathbf{w}}$ , given a dataset  $\mathcal{D}$  and a query vector  $\mathbf{w}$  is a symmetric binary matrix of size  $t \times t$  such that:

$$\Phi_{\mathbf{w}}[i, j] = \begin{cases} 1 & \text{if } \mathbf{w}[i] = \mathbf{w}[j]; \\ 0 & \text{otherwise.} \end{cases}$$

**Definition 7 (Result Pattern and Search Volume).** The result pattern (rp), denoted as  $\Pi_{\mathbf{w}}$ , given a dataset  $\mathcal{D}$  and a query vector  $\mathbf{w}$  is a binary matrix of size  $t \times N$  such that:

$$\Pi_{\mathbf{w}}[i, j] = \begin{cases} 1 & \text{if } \mathbf{w}[i] \in \mathcal{D}[j]; \\ 0 & \text{otherwise.} \end{cases}$$

The search volume (sv) of a keyword  $\mathbf{w}[i]$  can be derived from the result pattern as:  $\text{sv}(\mathbf{w}[i]) = \sum_{j=1}^N \Pi_{\mathbf{w}}[i, j]$ .

Intuitively, the search pattern reveals which queries have the same underlying keyword, the result pattern reveals which documents contain the queried keyword, and the search volume indicates the number of documents that match the queried keyword.

**Definition 8 (Trace).** The trace of  $H_t$  is the vector:

$$T(H_t) = (\text{id}(\mathcal{D}), N, m, \Phi_{\mathbf{w}}, \Pi_{\mathbf{w}}).$$

## 3 Models

**System Model.** Our system includes a user and a server. The user outsources its database, including encrypted documents, and an encrypted search index to support private keyword search and document update.

**Threat and Security Models.** We consider the standard threat model of DSSE, where the server can be corrupt while the user is honest [15]. We assume a semi-honest (honest but curious) adversary that adheres to the protocol but attempts to infer information about the user's queries and data (i.e., keyword search and document updates).

We are ready to define the adaptive semantic security for DSSE. Informally, the definition states that a DSSE scheme is secure if an adversary that observes the view can be simulated by an algorithm that only sees the trace; this implies that the trace captures all the information that is known by the adversary.

**Definition 9 (Adaptive Security of DSSE).** A DSSE scheme is *adaptively semantically secure* if for all  $t \in \mathbb{N}$  and for all (non-uniform) probabilistic polynomial time algorithm (the simulator)  $\mathcal{S}$  such that for all traces  $T_t$  of length  $t$ , all polynomial samplable distributions  $\mathcal{H}_t$  over  $\{H_t : T(H_t) = T_t\}$  (i.e., the set of histories with

trace  $T_t$ , all functions  $g : \{0, 1\}^{|H_t|} \rightarrow \{0, 1\}^{\text{poly}(|H_t|)}$ , all  $0 \leq s \leq t$ , and sufficiently large  $\lambda$ :

$$\left| \Pr[\mathcal{A}(V_\kappa^s(H_t)) = g(H_t^s)] - \Pr[\mathcal{S}(T(H_t^s)) = g(H_t^s)] \right| < \delta(\lambda),$$

where  $H_t \leftarrow \mathcal{H}_t$ ,  $(\kappa, \text{st}, \text{EDB}) \leftarrow \text{Setup}(1^\lambda)$ , and probabilities are taken over  $\mathcal{H}_t$  and the internal coins of  $\mathcal{A}, \mathcal{S}$  and the underlying  $\text{Setup}, \text{SrchTkn}, \text{Srch}, \text{UpdtTkn}, \text{Updt}$  algorithms.

We introduce a leakage function family  $\mathcal{L}_H = \{\mathcal{L}_H^{\text{Setup}}, \mathcal{L}_H^{\text{Srch}}, \mathcal{L}_H^{\text{Updt}}\}$  to control exactly the information of history  $H$  leaked during setup, search, and update, respectively. When an oracle is queried for the  $t$ -th operation, any function in  $\mathcal{L}_H$  is instantiated with  $H$  being the history consisting of the first  $(t-1)$  operations and with the  $t$ -th operation as a function input. It records the leakage incurred due to the last operation while taking all historical operations into consideration. We omit  $H$  if there is no ambiguity. Before any query (i.e.,  $H = \{\emptyset\}$ ),  $\mathcal{L} = \mathcal{L}^{\text{Setup}}$ .

**Definition 10 (Forward Privacy of DSSE).** An adaptively-secure DSSE is forward-private if the update leakage  $\mathcal{L}_H^{\text{Update}}(f, \mathcal{W}_f)$  of any update  $(f, \mathcal{W}_f)$  can be written as  $\mathcal{L}'(f)$ , where  $\mathcal{L}'$  is stateless.

Differential privacy (DP) [30] is a widely recognized formal framework for protecting privacy, and it has been adopted in various domains, such as database security [19, 79, 87, 89]. The notion is defined by a privacy parameter  $\epsilon$ , which guarantees that when two neighboring inputs are given to a DP algorithm, the probability of obtaining any particular output remains nearly the same. As a result, an observer cannot reliably infer sensitive information about the underlying input from the output. Here, we represent a DSSE scheme as a function  $\text{SE}$  that takes the user input (a dataset  $\mathcal{D}$  and a query vector  $\mathbf{w}$ ) and outputs the leakage (a trace  $T$ ). We borrow the definitions of differential privacy for documents and keywords of DSSE from [79], which imply search result pattern and search volume protection as follows.

**Definition 11 (Differential Privacy for Documents [79]).** A DSSE characterized by function  $\text{SE}$  provides  $\epsilon$ -DP for documents iff for any keyword list of length  $t$ ,  $\mathbf{w} \in W^t$ , for any pair of neighboring databases  $\mathcal{D}, \mathcal{D}' \in 2^{2^W}$  (they differ in exactly one position  $i$  and exactly one keyword  $w$ , i.e.,  $w$  is in either  $\mathcal{D}[i]$  or  $\mathcal{D}'[i]$  but not both), and any trace  $T$ , the following holds:

$$\Pr[\text{SE}(\mathcal{D}, \mathbf{w}) = T] \leq e^{\epsilon} \Pr[\text{SE}(\mathcal{D}', \mathbf{w}) = T]. \quad (1)$$

We refer to this notion “differential privacy for documents” instead of “for result and volume patterns”, since the hidden variable in this definition is the database that contains the documents,  $\mathcal{D}$ . Intuitively, satisfying the above definition guarantees that no one can determine if a document contains a keyword given the trace. This indicates that a DSSE that provides  $\epsilon$ -DP for documents also provides  $\epsilon$ -DP for result pattern and search volume since the similarity in result pattern implies the similarity in search volume (formally stated in Corollary 1). Note that document size leakage in search result is considered out-of-scope in our work.

**Definition 12 (Differential Privacy for Keywords [79]).** A DSSE characterized by function  $\text{SE}$  provides  $\epsilon$ -DP for keywords iff for any database  $\mathcal{D} \in 2^{2^W}$ , for any pair of neighboring keyword lists  $\mathbf{w}, \mathbf{w}' \in$

$W^{|\mathbf{w}|}$  ( $\mathbf{w}$  and  $\mathbf{w}'$  differ in only one element,  $\mathbf{w}[i] \neq \mathbf{w}'[i]$ ), and any trace  $T$ , the following holds:

$$\Pr[\text{SE}(\mathcal{D}, \mathbf{w}) = T] \leq e^{d \cdot \epsilon} \Pr[\text{SE}(\mathcal{D}, \mathbf{w}') = T]. \quad (2)$$

Here,  $d$  is the number of different documents between  $\mathcal{D}(\mathbf{w}[i])$  and  $\mathcal{D}(\mathbf{w}'[i])$ . A scheme that ensures differential privacy makes it difficult for an observer, even when considering the trace (permitted leakage), to distinguish whether the user’s query corresponds to one keyword list or the other. Consequently, this property ensures search pattern privacy, preventing the server from inferring whether two queries are associated with the same keyword.

## 4 Rerandomized PIR (RePIR)

PIR [22, 40, 45] enables private retrieval on a *public* database (owned by the server) without revealing the queried data. We propose a new PIR-based scheme for *private* database (encrypted and owned by the user) that supports output rerandomization, allowing a fresh random mask to be applied to the retrieved result. In other words, our scheme permits the server to obtain the content of the queried item that is decrypted and then re-encrypted with a new mask (without leaking the corresponding item index). We introduce RePIR, which stands for Rerandomized PIR, formally defined as follows.

### 4.1 Definitions

**Definition 13 (RePIR).** A RePIR scheme is a tuple of PPT algorithms defined as follows:

- $(\kappa, \text{st}, \text{EDB}) \leftarrow \text{Setup}(1^\lambda, \text{IDX}, N, m)$ : Given a security parameter  $\lambda$ , and an index  $\text{IDX}$  of size  $N \times m$ , it outputs a secret key  $\kappa$ , a state  $\text{st}$  and an encrypted database  $\text{EDB}$ .
- $(\text{qu}, \mathbf{r}) \leftarrow \text{Query}(v, \kappa, \text{st})$ : Given an index  $v \in [m]$ , a secret key  $\kappa$  and a state  $\text{st}$ , it outputs a query  $\text{qu}$  and a random mask  $\mathbf{r}$ .
- $\mathbf{d} \leftarrow \text{Answer}(\text{qu}, \text{EDB})$ : Given a query  $\text{qu}$  and an encrypted database  $\text{EDB}$ , it outputs a masked result  $\mathbf{d} = \text{IDX}[* , v] \boxplus \mathbf{r}$ .

**Correctness.** When the user and the server execute the RePIR protocol faithfully, the user should recover its desired data column with all but negligible probability in the implicit correctness parameter. We say that a RePIR scheme has *correctness error*  $\delta$  if, on index size  $N \times m$ , with  $N, m \in \mathbb{N}$ , for all indices  $\text{IDX} = [\mathbf{d}_1 \ \mathbf{d}_2 \ \dots \ \mathbf{d}_m]$ , and for all indices  $v \in [m]$ , the following probability is at least  $1 - \delta$ :

$$\Pr \left[ \begin{array}{l} (\kappa, \text{st}, \text{EDB}) \leftarrow \text{Setup}(1^\lambda, \text{IDX}, N, m) \\ \mathbf{d}_v \boxplus \mathbf{r} = \hat{\mathbf{d}}_v : \begin{array}{l} (\text{qu}, \mathbf{r}) \leftarrow \text{Query}(v, \kappa, \text{st}) \\ \hat{\mathbf{d}}_v \leftarrow \text{Answer}(\text{qu}, \text{EDB}) \end{array} \end{array} \right].$$

**Security.** The user’s query should reveal no information about its desired data column. That is, we say that a RePIR scheme is  $(T, \tilde{\epsilon})$ -secure if, for all adversaries  $\mathcal{A}$  running in time at most  $T$ , on index  $\text{IDX}$  of size  $N \times m$ , and for all  $v_1, v_2 \in \mathbb{N}$ :

$$\left| \Pr[\mathcal{A}(1^N, \text{qu}) = 1 : (\text{qu}, \_) \leftarrow \text{Query}(v_1, \kappa, \text{st})] - \Pr[\mathcal{A}(1^N, \text{qu}) = 1 : (\text{qu}, \_) \leftarrow \text{Query}(v_2, \kappa, \text{st})] \right| \leq \tilde{\epsilon}.$$

### 4.2 Our Concrete RePIR Scheme

We introduce our RePIR construction for private information retrieval, which supports fresh random masking for each query. Figure 1 presents our RePIR scheme. Given an index  $\text{IDX}$  of size  $N \times m$ ,

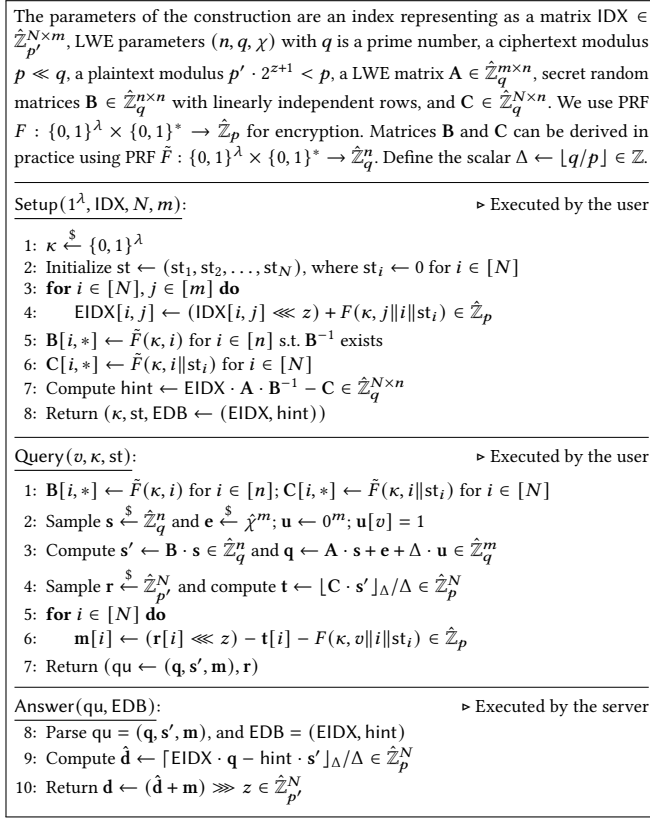


Figure 1: Our proposed RePIR scheme.

the user wishes to query for a column at index  $v \in [m]$ . The user builds a unit vector  $\mathbf{u} \in \hat{\mathbb{Z}}_2^m$  (i.e., the vector of all zeros with a single '1' at index  $v$ ), element-wise encrypts it using Regev's LWE-based encryption scheme (§2.1), and sends this encrypted vector to the server. The server computes the matrix-vector product between the index and the query vector, followed by unmasking the hint while also adding a random value to the final output. The added random value is fresh per query and can be precomputed by the user, setting the stage for later computation/processing steps on the new masked data if necessary.

We assume that  $\text{IDX} \in \hat{\mathbb{Z}}_{p'}^{N \times m}$  is encrypted to protect confidentiality. RePIR employs a pseudorandom function (PRF) function  $F$  whose output is in  $\hat{\mathbb{Z}}_p$  for encryption. Furthermore, RePIR needs to reserve a bit in the original data before encryption to avoid rounding error when evaluating the output answer (RePIR.Answer, lines 9–10). Note that  $\lfloor a + b \rfloor_\Delta / \Delta = a + \lfloor b \rfloor_\Delta / \Delta + e$ , where  $e \in \{0, 1\}$  is a small error. Thus, “reserving” a bit for the error in the data before being encrypted with PRF can “rule out” such error after unpacking the hint, rounding then adding the random mask in the later step. Assume that  $p' \cdot 2^{z+1} < p$ , with  $z = 1$ . For each  $\text{IDX}[i, j]$  with  $i \in [N]$  and  $j \in [m]$ , the user encrypts it using a pseudorandom function (PRF)  $F : \{0, 1\}^\lambda \times \{0, 1\}^* \rightarrow \hat{\mathbb{Z}}_p$  as:

$$\text{EIDX}[i, j] \leftarrow (\text{IDX}[i, j] \lll z) + F(\kappa, j \| i \| \text{st}_i) \pmod{p},$$

where  $\text{st}_i$  is the update state of the data in row  $i$  (a counter initialized with 0, and incremented by one for each update). The final encrypted index is  $\text{EIDX} \in \hat{\mathbb{Z}}_p^{N \times m}$ .

SimplePIR [45] uses Regev encryption to encrypt PIR queries, but the hint is stored on the user side, which is large in our case if applied for SSE because of the LWE parameter  $n$  (e.g.,  $n = 2048$ ), and also the server cannot unfold the hint since that reveals the queried data. As our output is masked with a random value, we propose a new approach to store the hint on the server. Let  $(n, q, \chi)$  be LWE parameters, where  $q$  is a prime number, and  $\mathbf{A}$  is a LWE matrix. Let  $\mathbf{B} \in \hat{\mathbb{Z}}_q^{n \times n}$  and  $\mathbf{C} \in \hat{\mathbb{Z}}_q^{N \times n}$  be two secret random matrices independent of the database that can be derived in practice using a PRF  $\tilde{F} : \{0, 1\}^\lambda \times \{0, 1\}^* \rightarrow \hat{\mathbb{Z}}_q^n$  as:  $\mathbf{B}[i, *] \leftarrow \tilde{F}(\kappa, i)$  for  $i \in [n]$ , and  $\mathbf{C}[i, *] \leftarrow \tilde{F}(\kappa, i \| \text{st}_i)$  for  $i \in [N]$ . Conditioned on  $\mathbf{B}$  having linearly independent rows over the prime field, its inverse  $\mathbf{B}^{-1}$  exists. Therefore, the user computes and outsources a hint to the server as

$$\text{hint} \leftarrow \text{EIDX} \cdot \mathbf{A} \cdot \mathbf{B}^{-1} - \mathbf{C} \in \hat{\mathbb{Z}}_q^{N \times n},$$

Let  $\mathbf{s} \xleftarrow{\$} \hat{\mathbb{Z}}_q^n$  and  $\mathbf{e} \xleftarrow{\$} \chi^m$ . The Regev encryption of a unit vector  $\mathbf{u} \in \hat{\mathbb{Z}}_p^m$  ( $\mathbf{u}[v] = 1$  and  $\mathbf{u}[j] = 0 \forall j \neq v$ ) is a vector:

$$\mathbf{q} = \text{Enc}(\mathbf{u}) \leftarrow (\mathbf{A} \cdot \mathbf{s} + \mathbf{e} + \lfloor q/p \rfloor \cdot \mathbf{u}) \pmod{q},$$

Matrix  $\mathbf{A}$  is a pseudorandom matrix and can be used to hide polynomially many secrets  $\mathbf{s}$ , provided that each query uses an independent vector  $\mathbf{s}$  and error vector  $\mathbf{e}$ . Also, the user computes the mask  $\mathbf{m}$  as follows:

$$\begin{aligned} \mathbf{t} &\leftarrow \lfloor \mathbf{C} \cdot \mathbf{s}' \rfloor_\Delta / \Delta \pmod{p}, \quad \mathbf{r} \xleftarrow{\$} \hat{\mathbb{Z}}_{p'}^N, \\ \mathbf{m}[i] &\leftarrow (\mathbf{r}[i] \lll z) - \mathbf{t}[i] - F(\kappa, v \| i \| \text{st}_i) \pmod{p}, \end{aligned}$$

for  $i \in [N]$ . Let  $\mathbf{s}' \leftarrow \mathbf{B} \cdot \mathbf{s} \in \hat{\mathbb{Z}}_q^n$ . Under the LWE assumption (§2.1), given  $(\mathbf{A}, \mathbf{A} \cdot \mathbf{s} + \mathbf{e})$ , recovering the secret  $\mathbf{s}$  is computationally hard. Also,  $\mathbf{B}$  is a secret matrix whose linearly independent rows form a basis that spans  $\hat{\mathbb{Z}}_q^n$ , therefore  $\mathbf{B} \cdot \mathbf{s} \stackrel{c}{\approx} \tilde{\mathbf{r}}$ , where  $\tilde{\mathbf{r}} \xleftarrow{\$} \hat{\mathbb{Z}}_q^n$ . Together with the encrypted query  $\mathbf{q}$ , the user encloses  $\mathbf{s}'$  and  $\mathbf{m}$  for decryption, removing the complementary hint incurred by  $\mathbf{C} \cdot \mathbf{s}'$ , and adding a random mask value  $\mathbf{r}$  simultaneously. The server uses the encrypted query to retrieve the queried data  $\text{EIDX}[:, v]$  but hidden by the complementary hint value  $\mathbf{C} \cdot \mathbf{s}'$ :

$$\begin{aligned} \hat{\mathbf{d}} &= \lceil \text{EIDX} \cdot \mathbf{q} - \text{hint} \cdot \mathbf{s}' \rceil_\Delta / \Delta \\ &= \lceil \text{EIDX} \cdot (\mathbf{A} \cdot \mathbf{s} + \mathbf{e} + \Delta \cdot \mathbf{u}) \\ &\quad - (\text{EIDX} \cdot \mathbf{A} \cdot \mathbf{B}^{-1}) \cdot \mathbf{B} \cdot \mathbf{s} + \mathbf{C} \cdot \mathbf{s}' \rceil_\Delta / \Delta \\ &= \lceil \text{EIDX} \cdot \Delta \cdot \mathbf{u} + \mathbf{C} \cdot \mathbf{s}' + \text{EIDX} \cdot \mathbf{e} \rceil_\Delta / \Delta \\ &= \text{EIDX}[:, v] + \lfloor \mathbf{C} \cdot \mathbf{s}' \rfloor_\Delta / \Delta + \mathbf{e}' \pmod{p}, \end{aligned}$$

with  $\mathbf{e}' \in \{0, 1\}^N$  and  $\mathbf{e}'[i] \in \hat{\mathbb{Z}}_p$ . After unpacking the hint, the server continues to remove the complementary component  $\lfloor \mathbf{C} \cdot \mathbf{s}' \rfloor_\Delta / \Delta$ , then obtains the retrieved data  $\text{IDX}[:, v]$  masked with a random value  $\mathbf{r}$  by adding  $\mathbf{m}$ :

$$\begin{aligned} \mathbf{d}[i] &= (\hat{\mathbf{d}}[i] + \mathbf{m}[i]) \ggg z \\ &= (\text{EIDX}[i, v] + \lfloor \mathbf{C} \cdot \mathbf{s}' \rfloor_\Delta / \Delta[i] + \mathbf{e}'[i] + (\mathbf{r}[i] \lll z) \\ &\quad - \lfloor \mathbf{C} \cdot \mathbf{s}' \rfloor_\Delta / \Delta[i] - F(\kappa, v \| i \| \text{st}_i)) \ggg z \\ &= ((\text{IDX}[i, v] \lll z) + (\mathbf{r}[i] \lll z) + \mathbf{e}'[i]) \ggg z \\ &= \text{IDX}[i, v] + \mathbf{r}[i] \pmod{p'}, \end{aligned}$$



|                                                                                                                                                                                                                                                                                                                   |                        |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| Setup( $1^\lambda$ ):<br>1: $\text{IDX} \leftarrow 0^{N \times m}$ , where $\text{IDX}[i, j] \in \mathbb{Z}_{p'}^\ell$ , for $i \in [N]$ and $j \in [m]$<br>2: $(\kappa, \text{st}, \text{EDB}) \leftarrow \text{RePIR.Setup}(1^\lambda, \text{IDX}, m, N)$<br>3: <b>return</b> $(\kappa, \text{st}, \text{EDB})$ | ▶ Executed by the user |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|

Figure 2: FROST setup algorithm.

for  $i \in [N]$ . RePIR outputs correctly as long as for all  $i \in [N]$ , the absolute value of the error  $\langle \text{EIDX}[i, *], \mathbf{e} \rangle$  is less than or equal to  $\Delta = \lfloor q/p \rfloor$ .

**Theorem 1 (RePIR).** *On database size  $N \times m$ , correctness failure probability  $\delta$ , adversary time  $T$ , and security failure probability  $\epsilon$ , let*

- $\chi$  be the discrete Gaussian distribution with variance  $\sigma^2$ ,
- $(n, q, \chi)$  be LWE parameters achieving  $(T, \epsilon)$ -security for  $m$  LWE samples, and
- $p \in \mathbb{N}$  and  $p' \in \mathbb{N}$  be a ciphertext and a plaintext modulus, respectively, chosen to satisfy  $p' \cdot 2^{z+1} < p$  and

$$\lfloor q/p \rfloor \geq \sqrt{2}/2 \cdot \sigma \cdot p \cdot \sqrt{m \cdot \ln(2/\delta)}. \quad (3)$$

Then, for a random LWE matrix  $\mathbf{A} \xleftarrow{\$} \hat{\mathbb{Z}}_q^{m \times n}$ , RePIR is a  $(T - O(m), \epsilon)$ -secure PIR scheme on database size  $N \times m$ , over plaintext space  $\hat{\mathbb{Z}}_{p'}$ , with correctness error  $\delta$ .

PROOF. See details in our full version [65, Appendix A].  $\square$

We prove RePIR security via query indistinguishability ([65, Lemma 1] gives single query and [65, Corollary 1] extends to  $Q$  queries via hybrid arguments). Therefore, this implies RePIR is simulation-based secure (SIM-secure), since RePIR query transcripts can be simulated from allowed leakage (e.g., public parameters  $N$  and  $m$ ).

## 5 Our Proposed FROST Scheme

Building on RePIR, we present our proposed FROST scheme.

### 5.1 Search Index Initialization

In FROST, the user builds an index for keyword-document representation. For high-level idea, we use a probabilistic data structure (i.e., BF) to create a compressed index. Suppose there are  $N$  documents, the user extracts a set of unique keywords  $\mathcal{W}_f$  for each document  $f \in [N]$  and computes its BF representation as  $\mathbf{u}_f \leftarrow \text{BF.Gen}(\mathcal{W}_f) \in \{0, 1\}^m$ . The search index contains  $N$  BF vectors, which is interpreted as an  $N \times m$  binary matrix  $\text{IDX} = [\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_N] \in \{0, 1\}^{N \times m}$ . Searching a keyword incurs checking BF membership by reading  $K$  columns  $\mathbf{d}_{H_1(w)}, \mathbf{d}_{H_2(w)}, \dots, \mathbf{d}_{H_K(w)}$  in  $\text{IDX}$ , where  $\mathbf{d}_{H_k(w)} = \text{IDX}[*, H_k(w)]$  for  $k \in [K]$ , with  $H_k$  is a mapping of BF and  $K$  is the BF parameter (defined in §2). The keyword  $w$  appears in document  $i \in [N]$  iff  $\sum_{k=1}^K \mathbf{d}_{H_k(w)}[i] = K$ . Updating a document  $f$  recreates a new BF representation of the updated keywords and overwrites the corresponding row  $\text{IDX}[f, *]$ .

The user can build a secure search system using RePIR with  $\text{IDX}$  as input index, similar to the existing work using PIR [27]. In particular, RePIR allows the user to identify query-matching documents without revealing them directly as the results are masked with random values. The user can then unmask these values to know which documents are matched, followed by using other techniques (e.g., ORAM) to retrieve the documents without exposing

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |                          |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|
| SrchTkn(st, $\kappa$ , w):<br>1: $\text{TPSet} \leftarrow \{\emptyset\}$ , $\text{FPSet} \leftarrow \{\emptyset\}$<br>2: <b>for</b> $i = 1$ to $N$ <b>do</b><br>▶ Sample $\text{TPSet}$ and $\text{FPSet}$ independently with probabilities $p$ and $q$ , resp.<br>3:   With probability $p$ : $\text{TPSet} \leftarrow \text{TPSet} \cup \{i\}$<br>4:   With probability $q$ : $\text{FPSet} \leftarrow \text{FPSet} \cup \{i\}$<br>5: Initialize $\text{mv} \leftarrow 0^N \in \mathbb{Z}_{p'}^N$<br>6: <b>for</b> $k = 1$ to $K$ <b>do</b><br>▶ Create a RePIR query for retrieving column $u_k$ with $k \in [K]$<br>7: $u_k \leftarrow H_k(w)$ , $(qu_k, r_k) \leftarrow \text{RePIR.Query}(u_k, \kappa, \text{st})$<br>▶ Create a transformation vector and compute the corresponding<br>▶ match value for each document, depending on whether the document<br>▶ is selected as a false positive, a true positive, or a random non-match<br>8: <b>for</b> $i = 1$ to $N$ <b>do</b><br>9: $\mathbf{v}_{\text{nm}} \leftarrow F^*(\kappa, u_k \  i \  \text{st}_i \  0) + \mathbf{r}_k[i] \in \mathbb{Z}_{p'}^\ell$<br>10: $\mathbf{v}_m \leftarrow F^*(\kappa, u_k \  i \  \text{st}_i \  1) + \mathbf{r}_k[i] \in \mathbb{Z}_{p'}^\ell$<br>11: <b>if</b> $i \in \text{FPSet}$ <b>then</b> ▶ If document $i$ is selected as a false positive<br>12:       Sample $y_{i,k} \xleftarrow{\$} \mathbb{Z}_{p'}$<br>13:       Compute $\mathbf{x}_{i,k} \in \mathbb{Z}_{p'}^\ell$ , s.t. $\langle \mathbf{v}_{\text{nm}}, \mathbf{x}_{i,k} \rangle = y_{i,k} \pmod{p'}$<br>14: <b>else if</b> $i \in \text{TPSet}$ <b>then</b> ▶ Else if document $i$ is selected as a true positive<br>15:       Sample $\mathbf{x}_{i,k} \xleftarrow{\$} \mathbb{Z}_{p'}^\ell$ ; and compute $y_{i,k} \leftarrow \langle \mathbf{v}_m, \mathbf{x}_{i,k} \rangle \pmod{p'}$<br>16: <b>else</b> ▶ Otherwise, document $i$ is selected as a random non-match<br>17:       Sample $\mathbf{x}_{i,k} \xleftarrow{\$} \mathbb{Z}_{p'}^\ell$ and $y_{i,k} \xleftarrow{\$} \mathbb{Z}_{p'}$<br>▶ Store transformation vectors and accumulate match values<br>18: $\text{tv}_k[i] \leftarrow \mathbf{x}_{i,k}$ ; $\text{mv}[i] \leftarrow \text{mv}[i] + y_{i,k} \pmod{p'}$<br>19: <b>return</b> $\mathbf{s} \leftarrow (\{(qu_k, \text{tv}_k)\}_{k \in [K]}, \text{mv})$ | ▶ Executed by the user   |
| Srch( $\mathbf{s}$ , $\text{EDB}$ ):<br>20: Parse $\mathbf{s} = (\{(qu_k, \text{tv}_k)\}_{k \in [K]}, \text{mv})$<br>21: <b>for</b> $k = 1$ to $K$ <b>do</b><br>▶ Retrieve the column w.r.t query $qu_k$ and obtain the rerandomized output<br>22: $\hat{\mathbf{d}}_k \leftarrow \text{RePIR.Answer}(qu_k, \text{EDB})$<br>23: <b>for</b> $i = 1$ to $N$ <b>do</b><br>▶ Compute dot product with the given transformation vector<br>24: $\mathbf{d}_k[i] \leftarrow \langle \hat{\mathbf{d}}_k[i], \text{tv}_k[i] \rangle \pmod{p'}$<br>25: <b>for</b> $k = 1$ to $K$ <b>do</b><br>26: $\mathbf{d} \leftarrow \sum_{k=1}^K \mathbf{d}_k \pmod{p'}$ ▶ Accumulate results over $K$ columns<br>27: $\mathcal{R} \leftarrow \{\emptyset\}$<br>28: <b>for</b> $i = 1$ to $N$ <b>do</b><br>29: <b>if</b> $\mathbf{d}[i] = \text{mv}[i]$ <b>then</b> $\mathcal{R} \leftarrow \mathcal{R} \cup \{i\}$ ▶ Check with the given match value<br>30: <b>return</b> $\mathcal{R}$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | ▶ Executed by the server |

Figure 3: FROST search algorithm.

the result pattern. In this work, we propose FROST, a new scheme that achieves single-round-trip keyword search while protecting search result pattern and search volume with differential privacy.

We first introduce a new approach in building the search index to facilitate obfuscating the results of keyword search via dot products as follows. We assume that  $\text{IDX}[i, j] \in \mathbb{Z}_{p'}^\ell$ , for  $i \in [N]$  and  $j \in [m]$  and  $F^*$  is a PRF with secret key  $\kappa$  as input and output a value in  $\mathbb{Z}_{p'}^\ell$ . A value at row  $i$  and column  $j$  in the search index is represented by an integer vector generated as  $\mathbf{v}_m \leftarrow F^*(\kappa, j \| i \| \text{st}_i \| 1) \in \mathbb{Z}_{p'}^\ell$ , if keyword  $j$  appears in document  $i$  (i.e., corresponding to match value 1), and  $\mathbf{v}_{\text{nm}} \leftarrow F^*(\kappa, j \| i \| \text{st}_i \| 0) \in \mathbb{Z}_{p'}^\ell$ , otherwise (i.e., corresponding to non-match value 0). In keyword search, the user sends transformation vectors  $\text{tv}$  to compute dot products with these state vectors (i.e.,  $\langle \mathbf{v}_m, \text{tv} \rangle$  or  $\langle \mathbf{v}_{\text{nm}}, \text{tv} \rangle$ ), collapsing them into scalar values that may belong to true positives, false positives, or random non-matches. Also, the user encloses a match value  $\text{mv}$  for the server to identify matched/unmatched documents, which corresponds to the true value if the document is a match, i.e.,  $\text{mv} = \langle \mathbf{v}_m, \text{tv} \rangle$ .

|                                                                                                                                                                         |                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|
| UpdTkn(st, $\kappa$ , $f$ , $\mathcal{W}_f$ ):                                                                                                                          | ▷ Executed by the user                     |
| 1: $\mathbf{u} \leftarrow \{0\}^m$ ; $\text{st}'_f \leftarrow \text{st}_f + 1$                                                                                          |                                            |
| 2: <b>for</b> $j = 1$ to $m$ <b>do</b>                                                                                                                                  |                                            |
| 3: $\mathbf{u}[j] \leftarrow F^*(\kappa, j \  f \  \text{st}'_f \  0) \in \mathbb{Z}_{p'}^\ell$                                                                         | ▷ Initialize an empty BF                   |
| 4: <b>for each</b> $w \in \mathcal{W}_f$ , $k = 1$ to $K$ <b>do</b>                                                                                                     | ▷ Insert keywords into BF                  |
| 5: $v_k \leftarrow H_k(w)$ , $\mathbf{u}[v_k] \leftarrow F^*(\kappa, v_k \  f \  \text{st}'_f \  1) \in \mathbb{Z}_{p'}^\ell$                                           |                                            |
| 6: <b>for</b> $j = 1$ to $m$ <b>do</b>                                                                                                                                  | ▷ Encrypt the updated row with PRF         |
| 7: $\mathbf{u}'[j] \leftarrow (\mathbf{u}[j] \ll z) + F(\kappa, j \  f \  \text{st}'_f) \in \mathbb{Z}_p^\ell$                                                          |                                            |
| 8: $\mathbf{B}[f, *] \leftarrow \tilde{F}(\kappa, f)$ for $f \in [N]$ ; $\mathbf{C}'[f, *] \leftarrow \tilde{F}(\kappa, f \  \text{st}'_f)$                             |                                            |
| 9: $\text{hint}'_f \leftarrow \mathbf{u}' \cdot \mathbf{A} \cdot \mathbf{B}^{-1} - \mathbf{C}'[f, *]$ , where $\text{hint}'_f[j] \in \mathbb{Z}_q^\ell$ for $j \in [n]$ | ▷ Update the corresponding row of the hint |
| 10: <b>return</b> $(\text{st}', \mathbf{u} \leftarrow (f, \mathbf{u}', \text{hint}'_f))$                                                                                |                                            |
| Updt(u, EDB):                                                                                                                                                           | ▷ Executed by the server                   |
| 11: Parse $\mathbf{u} = (f, \mathbf{u}', \text{hint}'_f)$ and EDB = (EIDX, hint)                                                                                        |                                            |
| 12: EIDX[f, *] $\leftarrow \mathbf{u}'$ , hint[f, *] $\leftarrow \text{hint}'_f$                                                                                        |                                            |
| 13: <b>return</b> EDB' $\leftarrow (\text{EIDX}, \text{hint})$                                                                                                          |                                            |

Figure 4: FROST update algorithm.

**Setup.** Figure 2 presents the setup procedure of FROST. The user initializes an empty search index IDX of size  $N \times m$ , where  $N$  is the number of documents and  $m$  is Bloom filter size, then executes RePIR.Setup algorithm with security parameter  $\lambda$ , IDX,  $N$ , and  $m$  as input to generate three components: a secret key  $\kappa$ , a local state  $\text{st}$ , and an encrypted database EDB. Note that Figure 2 follows the standard DSSE interface, where IDX is initialized as empty. If a full database is already available, IDX can instead be populated by document update, which is described in §5.3, for setting corresponding IDX entries during setup.

## 5.2 Keyword Search

Search result is the set of documents returned by the server in response to a query. In differentially private systems, the output may include documents that lack the target keyword or omit documents that do contain it. The impact of this inaccuracy can be described using two measurement metrics:

- **True Positive Rate (TPR):** the probability that the server returns a document that contains the queried keyword in a response to a query.
- **False Positive Rate (FPR):** the probability that the server returns a document that does *not* contain the requested keyword in a response to a query.

In most cases, the user aims for large TPR values (e.g., 0.95–0.99) and low FPR values (e.g., 0.025–0.2). We present the search protocol of FROST in Figure 3. To search for a keyword  $w$  in a database of  $N$  documents, the user computes its BF representation as column indices  $(u_1, u_2, \dots, u_k)$ , and creates corresponding RePIR queries for each  $u_k$  with  $k \in [K]$  (SrchTkn algorithm, lines 6–7). To execute a single-round-trip search while obfuscating search result patterns, the user first specifies which results should be true positives (TP, with high probability  $\text{TPR} \approx 0.95\text{--}0.99$ ), false positives (FP, with low probability  $\text{FPR} \approx 0.025\text{--}0.2$ ), or random non-matches (the remaining ones) (lines 1–4). Followed by creating RePIR queries for retrieving  $K$  columns, the user adds false positives and false negatives to the final output as follows. For each state vector  $(\mathbf{v}_m$  or  $\mathbf{v}_{nm})$  at the retrieved column  $k$  and row  $i$ , for all  $k \in [K]$  and  $i \in [N]$ , the user sends a transformation vector (tv)  $\mathbf{x}_{i,k} \in \mathbb{Z}_{p'}^\ell$ , depending on

whether document  $i$  belongs to true positives (TP), false positives (FP), or random non-matches set. For each document  $i \in \text{FP}$ , the user computes  $\mathbf{x}_{i,k}$  such that the dot products between  $\mathbf{x}_{i,k}$  with match value  $\mathbf{v}_m$  (i.e., value '1'), and non-match value  $\mathbf{v}_{nm}$  (i.e., value '0') are the same, i.e.,  $y_{i,k} \leftarrow \langle \mathbf{v}_m, \mathbf{x}_{i,k} \rangle = \langle \mathbf{v}_{nm}, \mathbf{x}_{i,k} \rangle \pmod{p'}$  (lines 11–13) as follows:

$$\mathbf{x}_{i,k}[j] \stackrel{\$}{\leftarrow} \mathbb{Z}_{p'}^\ell, \quad \text{for } j \in [\ell] \setminus \{l\},$$

$$\mathbf{x}_{i,k}[l] \leftarrow \frac{\sum_{j=1, j \neq l}^\ell (\mathbf{v}_m[j] - \mathbf{v}_{nm}[j]) \mathbf{x}_{i,k}[j]}{\mathbf{v}_{nm}[l] - \mathbf{v}_m[l]} \in \mathbb{Z}_{p'}^\ell,$$

where  $l \in [\ell]$  such that  $\mathbf{v}_{nm}[l] \neq \mathbf{v}_m[l]$ . The purpose is to make  $y_{i,k}$  the same regardless of the actual value in the search index, which can be either  $\mathbf{v}_m$  or  $\mathbf{v}_{nm}$ , thereby enforcing the document to appear in the search results. By contrast, for each document in TP, the user takes a random  $\mathbf{x}_{i,k}$  and computes  $y_{i,k} \leftarrow \langle \mathbf{v}_m, \mathbf{x}_{i,k} \rangle \pmod{p'}$ , which is the correct dot product value between match value  $\mathbf{v}_m$  and  $\mathbf{x}_{i,k}$  (lines 14–15). A document in the TP set is returned in the search result if it truly contains the queried keyword, which corresponds to the value  $\mathbf{v}_m$  in the search index. For the remaining documents that do not belong to either TP or FP, the user takes as random values for  $\mathbf{x}_{i,k}$  and  $y_{i,k}$ , such that these belong to random non-matches with a high probability (lines 16–17). To let the server know whether document  $i$  is a match or not, the user computes match value (mv) as  $\text{mv}[i] = \sum_{k=1}^K y_{i,k} \pmod{p'}$ , which is the summation value of  $y_{i,k}$  for  $K$  columns with  $i \in [N]$  (line 18). It is used on the server to check and return the matches as the search output (Srch algorithm, lines 25–29). Note that we can get a true positive if the document is correctly selected as a true positive (with probability  $p$ ), or when it is not but randomly selected as a false positive (with probability  $q$ ), i.e.,  $\text{TPR} = p + (1 - p)q = p + (1 - p)q$ . The probability of false positive is  $\text{FPR} = q$ . In our scheme, the value  $\ell$  is a security parameter, which is chosen to ensure that the probability of two duplicated transformation vectors is negligible. For example, with  $p'$  being a 24-bit prime number and  $\ell = 5$ , the probability of duplicates is  $1/2^{24(\ell-1)} = 2^{-96}$  (since  $\mathbf{x}_{i,k}[l]$  is not randomly selected for FP cases), which is negligible.

## 5.3 Document Update

FROST supports document update as other bitmap-based dynamic encrypted search schemes (e.g., [27, 48, 62]). We present the update procedure of FROST in Figure 4. Given an updated document with identifier  $f$  and a set of its keywords  $\mathcal{W}_f$ , the user increments the counter state value corresponding to document  $f$  as  $\text{st}'_f \leftarrow \text{st}_f + 1$  (UpdTkn algorithm, line 1). The user computes the new BF representation  $\mathbf{u} \in \mathbb{Z}_{p'}^\ell$  of the updated document with input keywords  $\mathcal{W}_f$  (lines 2–5). The user encrypts  $\mathbf{u}$  with PRF key  $\kappa$  and the incremented counter state value  $\text{st}'_f$  as  $\mathbf{u}'[j] \leftarrow (\mathbf{u}[j] \ll z) + F(\kappa, j \| f \| \text{st}'_f)$  (lines 6–7). Also, as the row corresponding to the updated document changes, the hint at row  $f$  also needs to be updated as  $\text{hint}'_f \leftarrow \mathbf{u}' \cdot \mathbf{A} \cdot \mathbf{B}^{-1} - \mathbf{C}'[f, *]$ , where  $\mathbf{C}'[f, *] \leftarrow \tilde{F}(\kappa, f \| \text{st}'_f)$  (lines 8–9). Finally, the user sends the update token  $\mathbf{u}$  including the document index  $f$ , the updated encrypted row  $\mathbf{u}'$ , and the updated hint  $\text{hint}'_f$  to the server to update its encrypted search



index and hint accordingly as  $\text{EIDX}[f, *] \leftarrow \mathbf{u}'$ , and  $\text{hint}[f, *] \leftarrow \text{hint}'_f$  (Updt algorithm, lines 11–12).

## 5.4 Optimization

We present several optimization strategies that can be applied to our FROST scheme to further improve its practicality.

### Reducing computation overhead by batching user queries.

Similar to PIR techniques [45, 49], we can apply “batch RePIR” to allow the user to fetch  $K$  columns at server-side cost  $\ll K \cdot m \cdot N$ , without increasing the hint size. The idea is to randomly partition the index of  $m$  columns into  $K$  chunks, each represented as a matrix of dimension  $N$ -by- $m/K$ . If the  $K$  columns that the user wants to fetch fall into distinct chunks, the server can obtain these columns by running RePIR once on each index chunk. This evenly-partitioned characteristic of BF is desirable in PIR settings, where the entire index must be traversed to prevent pattern leakage. BF can be tweaked to achieve this while maintaining the same FPR at the cost of a larger BF size  $m$  (see details in our full version [65, Appendix B]). However, with standard BF, more than one of the user’s desired columns may fall into the same chunk. In this case, the user can make  $\lambda$  RePIR queries to each of the  $K$  chunks to achieve the failure probability  $2^{-\Omega(\lambda)}$  [6, 49]. This optimization saves on server work as long as  $\lambda < K$ . However, it requires the user to send additional transformation vectors to eliminate the influence of redundant columns retrieved due to batching queries.

**Optimizing TPR and FPR.** The TPR and FPR can be palliated at the expense of an increase in the cost of the protocol: The TPR can be increased by replicating documents on the server side, dividing each document into  $L$  shards and requiring  $l < L$  shards for document recovery [19]. This incurs an extra communication, computation, and server storage cost. Likewise, false positives can be trivially filtered out by the user when checking whether or not a returned document contains the queried keyword. Thus, the effective cost of false positives is bandwidth consumption. As discussed earlier, existing SSE schemes balance privacy, performance, and utility in different ways. Schemes that leak search and result patterns often achieve high performance and utility, whereas stronger privacy solutions, such as ORAM or PIR, tend to introduce higher computation, communication, or user storage costs. We aim to design a middle-ground approach that obfuscates search result patterns with a trade-off between privacy and utility while remaining efficient in terms of computation and bandwidth costs.

## 5.5 Analysis

**5.5.1 Complexity.** We analyze the online asymptotic cost of FROST. We consider the number of bits of modulo  $p$ ,  $p'$ ,  $q$ , the LWE parameter  $n$ , parameter  $\ell$ , and the value of the BF parameter  $K$  as small constants. Let  $N$  be the number of documents. We implicitly include the cost for deriving the  $n \times n$  matrix  $\mathbf{B}$ , which is  $O(1)$  (as  $n$  is constant), and the cost for deriving matrix  $\mathbf{C}$ , which is  $O(N \cdot n) = O(N)$ , to the search and update cost of the user in the following analysis. To search for a keyword  $w$ , the user creates a query  $qu$  of size  $O(K \cdot (m + N + n) \cdot \ell) = O(m + N)$ . Also with each document, the user sends a transformation vector  $tv$  of size  $O(\ell \cdot K) = O(1)$ , and a match value  $mv$  of size  $O(1)$  for comparison.

Therefore, the overall user’s computation and bandwidth cost in search is  $O(m + N)$ .

For keyword search, the server incurs  $O(K \cdot N \cdot m \cdot \ell) = O(N \cdot m)$  modulo additions and multiplications for retrieval. The server performs  $O(K \cdot N \cdot \ell) = O(N)$  multiplications for dot products, followed by  $O(N)$  additions and comparisons to obtain the final matches. The overall server computation cost per search query is  $O(N \cdot m + N) = O(N \cdot m)$ .

To update a document, the user creates a new BF representation of size  $O(m \cdot \ell) = O(m)$ , followed by encrypting it, together with the hint of size  $O(n \cdot \ell) = O(1)$ . Thus, the total user’s bandwidth cost and computation cost per document update are both  $O(m)$ . The server does not incur any computation other than replacing the user’s database components (i.e., a row in the index and hint).

For storage, the user stores a secret key  $\kappa$  of size  $O(\lambda)$ , and matrices independent of the database that can be derived from a PRF with the secret key  $\kappa$  including: matrix  $\mathbf{B}$  of size  $O(n \cdot n \cdot \ell) = O(1)$ , and matrix  $\mathbf{C}$  of size  $O(N \cdot n \cdot \ell) = O(N)$ , and a LWE matrix  $\mathbf{A}$  of size  $O(m \cdot n \cdot \ell) = O(m)$ . The user stores a counter state  $st$  of size  $O(N)$ . The server stores an index of size  $O(K \cdot N \cdot m \cdot \ell) = O(N \cdot m)$ , and a hint of size  $O(N \cdot n \cdot \ell) = O(N)$ . In summary, the total server storage cost is  $O(N \cdot m)$ .

**5.5.2 Security Analysis.** We state the differential privacy for documents and keywords of FROST, implying search result pattern and search volume privacy, also the adaptive security of FROST when the underlying RePIR is SIM-secure as follows.

**Theorem 2.** FROST provides  $\epsilon$ -DP for documents and keywords, where  $\epsilon = \ln \left( \frac{\text{TPR}}{\text{FPR}} \frac{1 - \text{FPR}}{1 - \text{TPR}} \right)$ , with  $\text{TPR} = p + (1 - p)q$  and  $\text{FPR} = q$ .

Note that the resulting differential privacy parameter  $\epsilon$  of FROST is identical to that of OSSE. We adapt the differential privacy proof of OSSE [79, Appendix A] to prove our Theorem 2, with details given in our full version [65, Appendix C].

**Corollary 1.** FROST provides  $\epsilon$ -DP for documents and keywords, resulting in  $\epsilon$ -DP for search result pattern and search volume.

**Theorem 3.** FROST is adaptively semantically secure by Definition 9, and achieves forward privacy by Definition 10.

We adapt the semantic security proof of OSSE [79, Section VI] to prove our Theorem 3, with details provided in [65, Appendix D].

## 6 Experimental Evaluation

**Implementation.** We fully implemented all our proposed techniques in C++ consisting of about 1,200 lines of code. We used standard cryptographic libraries, including OpenSSL [1] for PRF and hash functions, libNTL [80] for modulo arithmetic. Our source code is available at: <https://github.com/vt-asaplab/FROST>.

**Hardware.** We used a server with 48-core Intel® Xeon®Platinum 8360Y CPU (2.40 GHz) and 512 GB RAM, running Rocky Linux 8.10.

**Parameters.** For LWE, we set the secret dimension  $n = 2048$ , use a 64-bit prime number modulus  $q = 18409157466922956641$ , set the error distribution  $\chi$  to be the discrete Gaussian distribution with standard deviation  $\sigma = 6.4$ , and allow correctness error  $\delta = 2^{-40}$  even with the largest BF size  $m = 11088$  (see §6.1.2). Our parameter selection ensures that the best-known attack against LWE requires

at least about  $2^{106}$  ring operations to break security, according to the modern lattice-attack-cost estimator [5]. We selected the BF size  $m$  to ensure less than 1 expected false positive, which is several orders of magnitude smaller than the number of false positives by DP, where each keyword hashes to  $K = 7$  BF indices (see details in our full version [65, Appendix B]). Therefore, the false positives observed in our evaluation come from the DP mechanism rather than BF. We selected plaintext modulus  $p' = 18483887$ , ciphertext modulus  $p = 80172943$ , and the dimension of state and transformation vectors  $\ell = 5$  as explained in §5.2.

**Datasets.** We used publicly available datasets to evaluate the performance of FROST, as well as its security against leakage abuse attacks in comparison with other SSE schemes using DP:

- *Enron* [3]: Enron email corpus, which contains about 500K emails of 150 employees from Enron corporation. We leveraged the same standard tokenization method described in Oblix [70]. We stemmed the words and removed stopwords and words that were  $>20$  or  $<4$  characters long or contained non-alphabetic characters.
- *Lucene* [2]: *Java-user* mailing list from the Lucene project, used by Cash et al. [14]. We took the emails of this mailing list from September 2001 until May 2020 (around 66,400 emails). Each email is one document in the dataset, and its keyword list is the set of words in the main body of the email that are part of an English dictionary, excluding English stopwords.
- *Wiki Dataset* [4]: *enwiki-latest-pages-article* dataset. We used WikiExtractor [7] to extract 18,607,516 distinct text-only articles from the dataset and removed empty articles that contain no keyword. We extracted unique keywords using the standard tokenization method.

**Attack-Resilience Evaluation.** We evaluated the resilience of FROST with state-of-the-art statistical-based leakage abuse attacks.

- *Freq* [67]: the frequency attack by Liu et al., which uses exclusively frequency information, and simply maps each query token to the keyword whose frequency is closest in Euclidean distance.
- *Jigsaw* [73]: Jigsaw is an attack by Nie et al., which works by first recovering the most distinctive queries that contain keywords with high volumes/frequencies, followed by leveraging co-occurrence information to further refine and recover the remaining queries. We used the original implementation and default parameters:  $\alpha = 0.1$ ,  $\beta = 0.9$ , BaseRec = 15, ConfRec = 10, and Refspeed = 5.
- *IHOP* [75]: IHOP is an attack by Oya and Kerschbaum that leverages volume and frequency leakage and a heuristic approach to solve quadratic assignment problem (QAP) via many iterations. IHOP takes advantage of frequency and volume co-occurrence information, which refers to how many documents two queries have in common. At each iteration, it fixes some assignments at random, and solves the linear assignment problem (LAP) with the remaining free assignments. We used the default parameters  $p_{\text{free}} = 0.25$  and  $n_{\text{iters}} = 1000$  as recommended by the authors.

We also evaluated against other attacks, including SAP [74] and GraphM attack [76] in our full version [65, Appendix E].

**Counterparts.** We compared FROST with state-of-the-art DP-based SSE schemes, including CLRZ [19] and OSSE [79] with their default parameters (see our Open Science section). To evaluate the effectiveness of these defenses to leakage abuse attacks, we used a subset of 30K documents from different datasets with varied

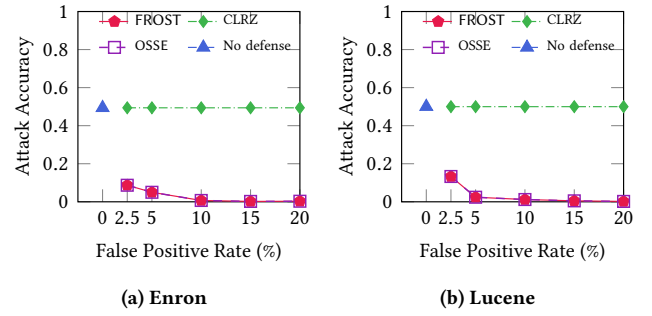


Figure 5: Accuracy of Freq attack ( $|W| = 1000$ ).

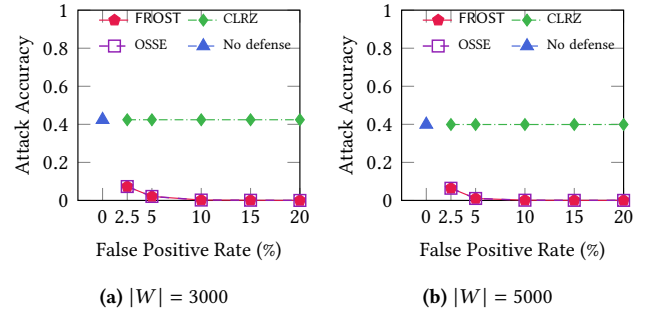


Figure 6: Accuracy of Freq attack (Wiki dataset).

keyword set sizes  $|W| \in \{1000, 3000, 5000\}$ , and varied FPRs in the range 0.025–0.2, while TPR = 0.95 is kept constant for all schemes. We split the dataset into two subsets: 20K documents for building the user database, and 10K documents for the adversary to use as auxiliary information. In addition, the attacks make use of query frequency information obtained from Google Trends as auxiliary data. For each attack, we report its accuracy, defined as the proportion of queries for which the underlying keywords are correctly recovered. To obtain stable and reliable results, we averaged the accuracy of each attack over 10 executions.

## 6.1 Results

**6.1.1 Accuracy of Attacks.** Figure 5 shows the accuracy of Freq attack on FROST and its counterparts over 100K queries with the same TPR 0.95 and varied FPRs increasing from 0.025 to 0.2. Figure 5a and Figure 5b are the evaluation on Enron and Lucene datasets, respectively, both with 1K keywords. Without defense (i.e., TPR = 1.0 and FPR = 0.0), the Freq attack achieves 49.4% accuracy on Enron, and 50.0% on Lucene. CLRZ cannot prevent this attack as it produces identical obfuscated outputs for the same keyword, thereby revealing search patterns. Consequently, the attack achieves the same accuracy when CLRZ is applied. FROST and OSSE effectively mitigate this attack since they output fresh obfuscation for repeated queries, where the attack accuracy is only 8.7%–0.2% on Enron, and 13.3%–0.2% on Lucene (lower is better). This demonstrates that applying clustering algorithms (e.g., k-means) to group similar result patterns does not weaken the differential privacy guarantees provided by FROST and OSSE in protecting search patterns. Figure 6 presents the attack accuracy on the Wiki dataset over 100K queries for larger database sizes, 3K and 5K keywords. Without any defense or when CLRZ is applied, the attack accuracies are 42.4%

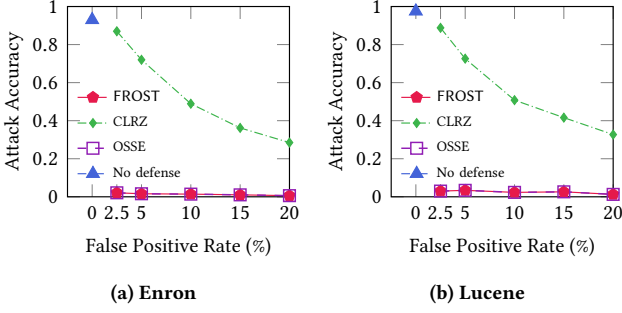
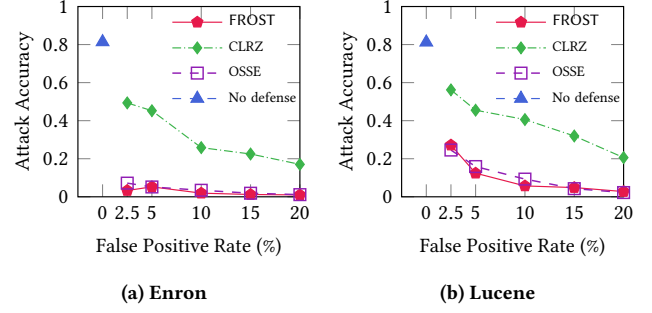
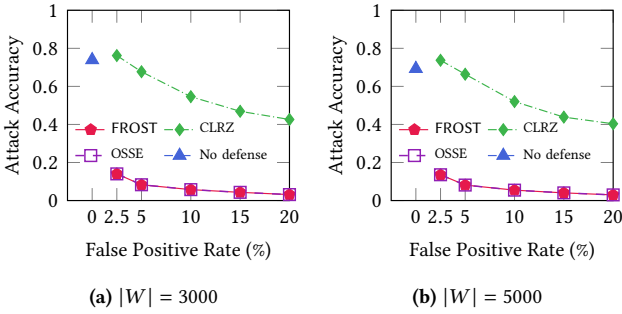
Figure 7: Accuracy of IHOP attack ( $|W| = 1000$ ).Figure 9: Accuracy of Jigsaw attack ( $|W| = 1000$ ).

Figure 8: Accuracy of IHOP attack (Wiki dataset).

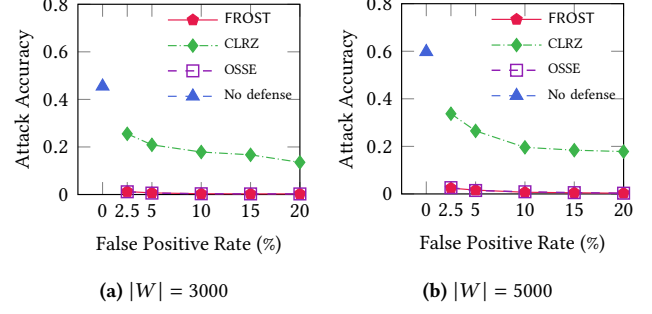


Figure 10: Accuracy of Jigsaw attack (Wiki dataset).

(3K keywords, Figure 6a) and 39.9% (5K keywords, Figure 6b). With FROST and OSSE, only 7.3%–0.1%, and 6.4%–0.1% of queries can be recovered correctly for 3K and 5K keywords, respectively.

Figure 9 presents the accuracy of Jigsaw attack over 25K queries. Without obfuscation, the attack can recover queried keywords with accuracy 81.4% and 81.2% for Enron and Lucene, respectively. For Enron dataset (Figure 9a), the attack accuracy drops from 3.3% to 1.0% for FROST, 7.2%–1.1% for OSSE, and 49.3%–17.0% for CLRZ. For Lucene dataset (Figure 9b), the corresponding figures are 27.2%–2.7%, 24.6%–2.2%, and 56.2%–20.6% for FROST, OSSE, and CLRZ, respectively. FROST and OSSE perform very closely overall, in which differences are insignificant and not monotonic in FPR since the differential privacy parameter  $\epsilon$  is the same in both works. As mentioned in [73], Jigsaw is good at recovering high-volume queries, which is about 10% of data per Zipfian law. It is explained in [73] that the accuracy of the attack can be negatively affected for data that does not follow Zipfian law, e.g., by padding random noise. Furthermore, the attack accuracy mostly relies on the precise recovery of distinctive queries, i.e., queries with distinguished frequency/volume. Figure 10 shows the accuracy of the Jigsaw attack on Wiki dataset with 3K keywords over 90K queries, and 5K keywords over 150K queries. Per Figure 10, without any defense, Jigsaw attack achieves accuracy 45.5% and 59.7% for 3K and 5K keywords, respectively. FROST and OSSE can significantly reduce these numbers, which decline down to 1.2%–0.3% for FROST, 1.2%–0.2% for OSSE (Figure 10a), and 2.4%–0.3% for FROST, 2.6%–0.3% for OSSE (Figure 10b), while the corresponding figures of CLRZ are 25.5%–13.5% and 33.7%–17.8%. This emphasizes the importance of protecting result patterns with fresh obfuscation to also obfuscate search patterns, which can only be achieved with FROST and OSSE, but cannot with CLRZ.

Figure 7 shows the accuracy of IHOP attack. Due to the high computational complexity of this attack, we tested with a smaller number of queries, which is 10K queries with 1K keywords. Without defense, the attack achieves accuracy 93.0% for Enron (Figure 7a) and 97.5% for Lucene (Figure 7b). For Enron dataset, the accuracy of the attack decreases from 2.1% down to 0.6% with FROST and OSSE, and 87.0%–28.5% with CLRZ. For Lucene dataset, the corresponding figures of the attack are 3.0%–1.3% for FROST and OSSE, and 88.8%–32.7% for CLRZ. DP-based approaches can mitigate the effectiveness of IHOP attack since it can obfuscate result patterns, falsifying the volume co-occurrence leakage information. Moreover, FROST and OSSE also obfuscate search patterns, reducing the impact of query frequency leakage. Figure 8 shows the accuracy of IHOP attack on Wiki dataset with significantly large numbers of queries and keywords, 30K queries with 3K keywords (Figure 8a), and 50K queries with 5K keywords (Figure 8b). Without defense, IHOP attack can precisely recover 73.9% of queries with database including 3K keywords, and 69.3% with 5K keywords. However, FROST and OSSE can reduce the attack accuracy down to 14.0%–3.1% with 3K keywords, and to 13.6%–3.0% with 5K keywords, while the corresponding figures of CLRZ are 76.2%–42.6%, and 73.7%–40.4%, respectively. We notice that under FROST and OSSE defenses, increasing the number of queries leads to lower accuracy and requires much longer attack execution time.

**6.1.2 Keyword Search.** For keyword search, we only used 8 CPU cores for evaluation. Figure 11 shows the total search delay and user bandwidth with the Enron dataset, where each email has an average of 73.2 keywords. We set TPR = 0.95 and FPR = 0.1. For the number of documents increasing from  $2^{10}$  to  $2^{19}$ , we selected the corresponding BF sizes  $m \in \{2912, 3745, 4676, 5810, 7273, 8099\}$ .

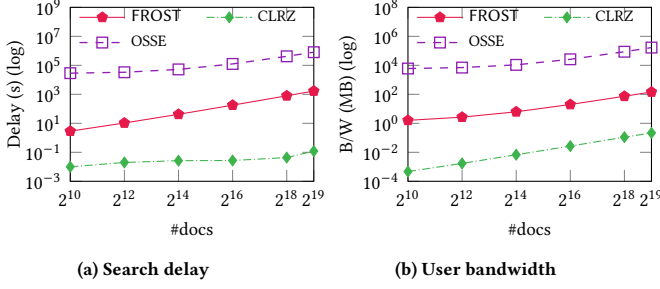


Figure 11: Total search delay and bandwidth (Enron, FPR = 0.1).

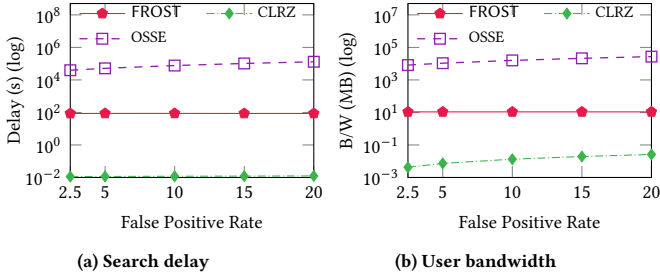


Figure 12: Total search delay and bandwidth (Enron, 32K documents).

Figure 11a presents the total delay to execute a keyword search, in which FROST takes 2.9s–1683.6s, OSSE takes 28798.8s–806757.0s, and CLRZ takes 10.0ms–120.8ms. FROST is  $479.2\times$ – $10012.6\times$  faster than OSSE while providing the same security guarantee. Although CLRZ is more efficient, it does not produce fresh obfuscations for repeated queries on the same keyword, thereby leaking search patterns. Figure 11b demonstrates the user bandwidth, including the query and response size for a keyword search. The communication cost of FROST increases from 1.6 MB to 144.9 MB, while that of OSSE is 5.9 GB–163.9 GB, which is  $1157.9\times$ – $3747.9\times$  larger.

Figure 12 shows the total search delay and user bandwidth for a subset of 32K documents with TPR = 0.95 and varied FPRs, increasing from 0.025 to 0.2. While FROST and CLRZ remain nearly unchanged, OSSE incurs significantly more delay and communication cost when increasing FPR. In particular, for Figure 12a, while the delay of OSSE rises sharply from 39466.2s to 130980.0s, that of FROST just slightly increases 86.1s–86.7s, and 11.2ms–12.5ms for CLRZ. Regarding communication cost (Figure 12b), OSSE incurs 8.0 GB–26.5 GB, whereas the bandwidth overhead of FROST is around 10.8 MB, and that of CLRZ is 4.3 KB–26.6 KB. The reason for the overhead of OSSE is when increasing FPR, the user must send additional search tokens, requiring extra computation for search, thereby incurring higher latency and bandwidth cost.

We also executed the same experiments with the Wiki dataset. In particular, we excluded documents containing no keywords, and randomly sampled 500K documents. For our extracted subset, each document has an average of 100.3 keywords. For the number of documents increasing from  $2^{10}$  to  $2^{19}$ , we selected the corresponding BF sizes  $m \in \{3983, 5124, 6398, 7966, 9954, 11088\}$ . Figure 13 illustrates the total search delay and bandwidth overhead with varied numbers of documents. For Figure 13a, FROST, OSSE, and CLRZ correspondingly take 2.9s–1762.2s, 28819.8s–810216.6s,

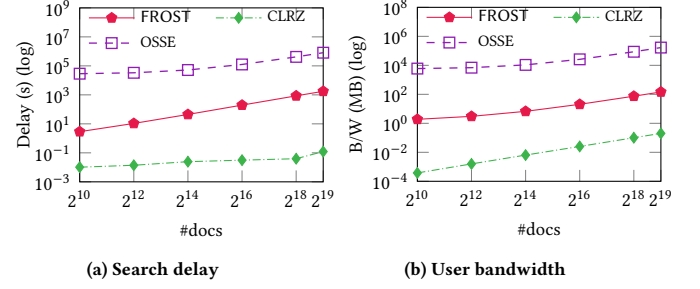


Figure 13: Total search delay and bandwidth (Wiki, FPR = 0.1).

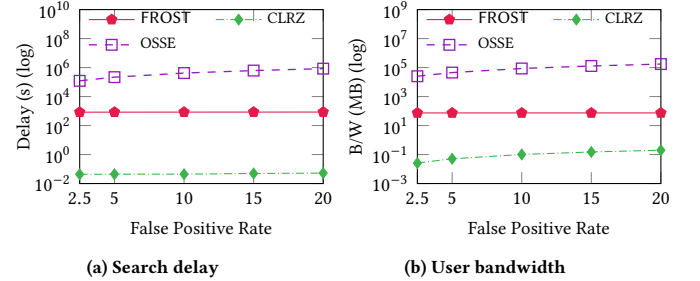


Figure 14: Total search delay and bandwidth (Wiki, 256K documents).

10.2ms–121.9ms to execute a keyword search. FROST is  $459.8\times$ – $9988.5\times$  faster than OSSE and  $281.9\times$ – $14456.7\times$  slower than CLRZ. For Figure 13b, the communication costs of FROST, OSSE, and CLRZ are 1.9 MB–145.7 MB, 5.8 GB–163.9 GB, and 0.4 KB–206.4 KB, respectively. FROST incurs  $1151.9\times$ – $3161.9\times$  smaller bandwidth overhead than OSSE, while requiring an additional 1.9 MB–145.5 MB compared to CLRZ as a trade-off for protecting the search patterns. Figure 14 presents the total search delay and user bandwidth for a subset of 256K documents with varied FPRs. While FROST and CLRZ remain virtually unchanged, in which FROST takes 839.0s–854.6s, and CLRZ takes 43.2ms–52.7ms, the total search delay of OSSE increases from 122188.8s–843029.4s (Figure 14a). Also, the bandwidth overhead of OSSE rises from 24.7 GB–170.5 GB, while that of FROST and CLRZ is 74.2 MB–74.4 MB, and 26.5 KB–205.2 KB, respectively (Figure 14b).

**Privacy–utility trade-offs and parameter insights.** For repeated queries, FROST generates fresh obfuscation each time because TPSet and FPSet are sampled anew per query, so repeated executions of the same keyword search do not return the same noisy result patterns, which is the same as OSSE, and stronger than CLRZ. For closely related or correlated queries, the privacy–utility trade-off is more application-dependent and depends on user behavior and database characteristics in the concrete deployment. For the evaluation in this section, we selected TPR and FPR to make existing attacks close to nearly random-guessing. Intuitively, with DP-based SE, larger false negative rate (i.e., lower TPR) affects utility and bandwidth, while larger FPR only increases extra returned documents and bandwidth. Unlike OSSE, FROST’s query size and server computation are independent of TPR and FPR.

**6.1.3 Document Update.** Due to space constraints, we present the evaluation for document update of FROST in our full version [65, Section 6.1.3].

## 7 Related Work

**SSE.** SSE was first introduced and formalized by Song et al. [81]. SSE enables encrypted queries to be executed over encrypted data using an encrypted index, while supporting enhanced security properties like forward privacy [12, 59], backward privacy [13, 39, 84, 85], efficiency improvements [17, 28], and extended query capabilities [56], as well as support for updates [16, 44, 52, 83]. Despite these benefits, many SSE schemes are susceptible to statistical inference attacks [14, 42, 54, 75, 76, 94], which often exploit inherent leakage patterns such as search [58, 67, 74], result [50], volume [60], and file-injection [96, 97]. Xu et al. [94] demonstrated that even schemes with strong security guarantees, such as forward and backward privacy, still remain vulnerable to leakage-abuse attacks.

**Multi-User SSE.** While SSE primarily supports single-user access, several approaches have been developed to extend functionality to multi-user scenarios [18, 55]. For instance, Chamani et al. [18] introduced a multi-user SSE model that incorporates data and query policy enforcement, along with verifiability through blockchain technology. The scheme in [55] allows users to query encrypted data without direct interaction with the data owner, relying instead on helper users. Other solutions either assume all users are trusted [27] or involve resource-intensive cryptographic mechanisms like MPC [33, 51, 62]. The scheme in [51] employs two independent custodians running a garbled circuit protocol to create search tokens, though it still reveals typical SSE leakage such as search, volume, and result patterns. Leakage mitigation techniques [38] can reduce some exposure but are mainly suited for single-user settings due to the high overhead of index rebuilding. Applying such techniques in multi-user contexts, where readers and writers are distinct, requires writers to monitor readers' query activities to maintain security. Multi-key SSE schemes [55, 93] offer decentralized access across users, yet they do not eliminate pattern leakage.

**PKSE.** PKSE schemes [8, 11, 31, 95] enable support for multiple writers; however, they do not ensure forward privacy, making them susceptible to file-injection attacks [97]. While hybrid model [90] can provide forward privacy, they require each writer to be actively involved in periodically regenerating encrypted tokens. Furthermore, many PKSE approaches are prone to be vulnerable by KGA. Some PKSE schemes can prevent KGA, but they depend on the presence of a trusted third party [57, 66, 91], or trusted setup [64].

**Oblivious storage systems.** Oblivious storage platforms make use of ORAM and/or PIR techniques to conceal search and result patterns during data operations such as retrieval or sharing [9, 20, 21, 23, 27, 68, 69], and keyword search [29, 33, 37, 48, 63, 72]. DP-based approach [19] can obfuscate result patterns, but still leaks search patterns or comes at the cost of significant computation and communication overhead to also hide search patterns [79].

**Hardware-assisted SE.** Trusted hardware technologies, such as Intel SGX [24], have been employed to develop efficient oblivious platforms supporting a range of tasks, including keyword search [36, 47, 70], SQL queries [32, 35], data storage [26, 46], and oblivious memory [86]. These systems rely on specific hardware security guarantees, such as isolation, resistance to tampering, and protection against side-channel attacks.

## 8 Conclusion

In this paper, we proposed and designed FROST, a new SSE scheme that conceals search patterns and obfuscates result patterns to mitigate leakage-abuse query-recovery attacks while maintaining high efficiency and requiring only a single communication round for keyword search. Compared with OSSE, the state-of-the-art SSE design with DP, FROST offers a significant advance in terms of concrete performance. In addition, we introduced RePIR, a new primitive for private information retrieval on a private database that supports output rerandomization, and also a novel approach for applying DP in SSE by using state and transformation vectors.

## Acknowledgment

We would like to thank the shepherd and anonymous reviewers for their insightful comments and constructive feedback on improving the quality of this work. This project was supported in part by NSF grant CNS-2350215 and the Commonwealth Cyber Initiative (CCI), an investment in the advancement of cyber R&D, innovation, and workforce development. For more information about CCI, visit [www.cyberinitiative.org](http://www.cyberinitiative.org).

## Ethics Considerations

We have carefully considered the ethics following the conference guideline. All datasets utilized in our experiments are publicly available and do not contain any harmful content. Our experiments were conducted exclusively within rigorously controlled environments, ensuring no disruption to other public systems. We believe our research was done ethically and complies with the ethics guidelines.

## Open Science

Our FROST implementation is available at: <https://github.com/vt-asaplab/FROST>. The repository includes a file README.md providing detailed instructions for setup, building the project, dataset extraction, and program execution.

We evaluated the effectiveness of DP-based approaches including our FROST, CLRZ [19], and OSSE [79] against the Freq [67], IHOP [75], SAP [74], and GraphM [76] attacks using the implementation available at <https://github.com/simon-oya/USENIX22-ihop-code/>, and against the Jigsaw [73] attack using the implementation at <https://github.com/JigsawAttack/JigsawAttack/>. For completeness, we also included the implementation of these attacks in our repository. For evaluating keyword search performance, we reported results based on the publicly available artifacts of OSSE: <https://github.com/z6shang/OSSE/>, and CLRZ: <https://github.com/donnod/APOSSE/>. The CLRZ implementation instantiates its underlying SSE using the RR2Lev scheme from the open-source SSE library Clusion <https://github.com/encryptedsystems/Clusion/>.

## References

- [1] OpenSSL: Cryptography and SSL/TLS Toolkit. <https://www.openssl.org>.
- [2] Apache Foundation. Mail archives of lucene. [https://mail-archives.apache.org/mod\\_mbox/lucene](https://mail-archives.apache.org/mod_mbox/lucene), 1999.
- [3] CMU William W. Cohen, MLD. Enron email datasets. <https://www.cs.cmu.edu/~enron>, 2015.
- [4] The wikipedia corpus. <https://dumps.wikimedia.org/>, 2022.
- [5] Martin Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *Journal of Mathematical Cryptology*, 9, 10 2015.



- [6] Sebastian Angel, Hao Chen, Kim Laine, and Srinath Setty. Pir with compressed queries and amortized query processing. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 962–979, 2018.
- [7] Giuseppe Attardi. Wikiextractor. <https://github.com/attardi/wikiextractor>, 2015.
- [8] Adam J. Aviv, Michael E. Locasto, Shaya Potter, and Angelos D. Keromytis. Ssaes: Secure searchable automated remote email storage. In *Twenty-Third Annual Computer Security Applications Conference (ACSAC 2007)*, pages 129–139, 2007.
- [9] Erik-Oliver Blass, Travis Mayberry, Guevara Noubir, and Kaan Onarlioglu. Toward robust hidden volumes using write-only oblivious ram. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security, CCS '14*, page 203–214, New York, NY, USA, 2014. Association for Computing Machinery.
- [10] Burton H Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426, 1970.
- [11] Dan Boneh, Giovanni Di Crescenzo, Rafail Ostrovsky, and Giuseppe Persiano. Public key encryption with keyword search. Cryptology ePrint Archive, Paper 2003/195, 2003. <https://eprint.iacr.org/2003/195>.
- [12] Raphael Bost.  $\Sigma$  o $\phi$ oc: Forward secure searchable encryption. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, CCS '16*, page 1143–1154, New York, NY, USA, 2016. Association for Computing Machinery.
- [13] Raphael Bost, Brice Minaud, and Olga Ohrimenko. Forward and backward private searchable encryption from constrained cryptographic primitives. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS '17*, pages 1465–1482, 2017.
- [14] David Cash, Paul Grubbs, Jason Perry, and Thomas Ristenpart. Leakage-abuse attacks against searchable encryption. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security, CCS '15*, page 668–679, New York, NY, USA, 2015. Association for Computing Machinery.
- [15] David Cash, Joseph Jaeger, Stanislaw Jarecki, Charanjit S. Jutla, Hugo Krawczyk, Marcel-Catalin Rosu, and Michael Steiner. Dynamic searchable encryption in very-large databases: Data structures and implementation. *IACR Cryptol. ePrint Arch.*, 2014:853, 2014.
- [16] David Cash, Stanislaw Jarecki, Charanjit Jutla, Hugo Krawczyk, Marcel-Cătălin Rosu, and Michael Steiner. Highly-scalable searchable symmetric encryption with support for boolean queries. In *Annual cryptography conference*, pages 353–373. Springer, 2013.
- [17] Javad Ghareh Chamani, Dimitrios Papadopoulos, Mohammadamin Karbasforushan, and Ioannis Demertzis. Dynamic searchable encryption with optimal search in the presence of deletions. In Kevin R. B. Butler and Kurt Thomas, editors, *31st USENIX Security Symposium, USENIX Security 2022*, pages 2425–2442. USENIX Association, 2022.
- [18] Javad Ghareh Chamani, Yun Wang, Dimitrios Papadopoulos, Mingyang Zhang, and Rasool Jalili. Multi-user dynamic searchable symmetric encryption with corrupted participants. *IEEE Transactions on Dependable and Secure Computing*, 20(1):114–130, 2023.
- [19] Guoxing Chen, Ten-Hwang Lai, Michael K. Reiter, and Yinqin Zhang. Differentially private access patterns for searchable symmetric encryption. In *IEEE INFOCOM 2018 - IEEE Conference on Computer Communications*, pages 810–818, 2018.
- [20] Weikeng Chen, Thang Hoang, Jorge Guajardo, and Attila A. Yavuz. Titanium: A metadata-hiding file-sharing system with malicious security. Cryptology ePrint Archive, Paper 2022/051, 2022. <https://eprint.iacr.org/2022/051>.
- [21] Weikeng Chen and Raluca Ada Popa. Metal: a metadata-hiding file-sharing system. In *NDSS Symposium 2020*, 2020.
- [22] Benny Chor, Eyal Kushilevitz, Oded Goldreich, and Madhu Sudan. Private information retrieval. *J. ACM*, 45(6):965–981, November 1998.
- [23] Sherman SM Chow, Katharina Feh, Russell WF Lai, and Giulio Malavolta. Multi-client oblivious ram with poly-logarithmic communication. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 160–190. Springer, 2020.
- [24] Victor Costan and Srinivas Devadas. Intel SGX explained. *Cryptology ePrint Archive*, 2016.
- [25] Reza Curtmola, Juan Garay, Seny Kamara, and Rafail Ostrovsky. Searchable symmetric encryption: Improved definitions and efficient constructions. In *Proceedings of the 13th ACM Conference on Computer and Communications Security, CCS '06*, page 79–88, New York, NY, USA, 2006. Association for Computing Machinery.
- [26] Emma Dauterman, Vivian Fang, Ioannis Demertzis, Natacha Crooks, and Raluca Ada Popa. Snoopy: Surpassing the scalability bottleneck of oblivious storage. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, pages 655–671, 2021.
- [27] Emma Dauterman, Eric Feng, Ellen Luo, Raluca Ada Popa, and Ion Stoica. Dory: An encrypted search system with distributed trust. In *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation, OSDI'20*, USA, 2020. USENIX Association.
- [28] Ioannis Demertzis, Javad Ghareh Chamani, Dimitrios Papadopoulos, and Charalampos Papamanthou. Dynamic searchable encryption with small client storage. 01 2020.
- [29] Ioannis Demertzis, Dimitrios Papadopoulos, and Charalampos Papamanthou. Searchable encryption with optimal locality: Achieving sublogarithmic read efficiency. In *Advances in Cryptology – CRYPTO 2018: 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19–23, 2018, Proceedings, Part I*, page 371–406, Berlin, Heidelberg, 2018. Springer-Verlag.
- [30] Cynthia Dwork. Differential privacy: A survey of results. In *Theory and Applications of Models of Computation—TAMC*, volume 4978 of *Lecture Notes in Computer Science*, pages 1–19. Springer Verlag, April 2008.
- [31] Nabeil Eltayieb, Rashad Elhabob, Alzubair Hassan, and Fagen Li. An efficient attribute-based online/offline searchable encryption and its application in cloud-based reliable smart grid. *Journal of Systems Architecture*, 98:165–172, 2019.
- [32] Saba Eskandarian and Matei Zaharia. Oblidb: Oblivious query processing for secure databases. *Proceedings of the VLDB Endowment*, 13(2), 2019.
- [33] Brett Hemenway Falk, Steve Lu, and Rafail Ostrovsky. Durasift: A robust, decentralized, encrypted database supporting private searches with complex policy controls. In *Proceedings of the 18th ACM Workshop on Privacy in the Electronic Society, WPES'19*, page 26–36, New York, NY, USA, 2019. Association for Computing Machinery.
- [34] Ben Fisch, Arthur Lazzaretti, Zeyu Liu, and Charalampos Papamanthou. Thorpir: Single server pir via homomorphic thorp shuffles. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS '24*, page 1448–1462, New York, NY, USA, 2024. Association for Computing Machinery.
- [35] Benny Fuhry, Jayanth Jain H. A, and Florian Kerschbaum. Encddb: Searchable encrypted, fast, compressed, in-memory database using enclaves. In *51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN 2021, Taipei, Taiwan, June 21–24, 2021*, pages 438–450. IEEE, 2021.
- [36] Benny Fuhry, Raad Bahmani, Ferdinand Brasser, Florian Hahn, Florian Kerschbaum, and Ahmad-Reza Sadeghi. Hardidx: Practical and secure index with sgx. In *IFIP Annual Conference on Data and Applications Security and Privacy*, pages 386–408. Springer, 2017.
- [37] Sanjam Garg, Payman Mohassel, and Charalampos Papamanthou. Tworam: Efficient oblivious ram in two rounds with applications to searchable encryption. In *Proceedings, Part III, of the 36th Annual International Cryptology Conference on Advances in Cryptology – CRYPTO 2016 - Volume 9816*, page 563–592, Berlin, Heidelberg, 2016. Springer-Verlag.
- [38] Marilyn George, Seny Kamara, and Tarik Moataz. Structured encryption and dynamic leakage suppression. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021*, pages 370–396, Cham, 2021. Springer International Publishing.
- [39] Javad Ghareh Chamani, Dimitrios Papadopoulos, Charalampos Papamanthou, and Rasool Jalili. New constructions for forward and backward private symmetric searchable encryption. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS '18*, page 1038–1055, New York, NY, USA, 2018. Association for Computing Machinery.
- [40] Niv Gilboa and Yuval Ishai. Distributed point functions and their applications. In *EUROCRYPT*, 2014.
- [41] Oded Goldreich and Rafail Ostrovsky. Software protection and simulation on oblivious rams. *J. ACM*, 43(3):431–473, May 1996.
- [42] Zichen Gui, Kenneth G. Paterson, and Sikhar Patranabis. Rethinking searchable symmetric encryption. Cryptology ePrint Archive, Paper 2021/879, 2021. <https://eprint.iacr.org/2021/879>.
- [43] Zichen Gui, Kenneth G. Paterson, and Tianxin Tang. Security analysis of MongoDB queryable encryption. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 7445–7462, Anaheim, CA, August 2023. USENIX Association.
- [44] Florian Hahn and Florian Kerschbaum. Searchable encryption with secure and efficient updates. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security, CCS '14*, page 310–320, New York, NY, USA, 2014. Association for Computing Machinery.
- [45] Alexandra Henzinger, Matthew M. Hong, Henry Corrigan-Gibbs, Sarah Meiklejohn, and Vinod Vaikuntanathan. One server for the price of two: Simple and fast Single-Server private information retrieval. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 3889–3905, Anaheim, CA, August 2023. USENIX Association.
- [46] Thang Hoang, Rouzbeh Behnia, Yeongjin Jang, and Attila A Yavuz. Mose: Practical multi-user oblivious storage via secure enclaves. In *Proceedings of the Tenth ACM Conference on Data and Application Security and Privacy*, pages 17–28, 2020.
- [47] Thang Hoang, Muslum Ozgur Ozmen, Yeongjin Jang, and Attila A Yavuz. Hardware-supported oram in effect: Practical oblivious search and update on very large dataset. *Proceedings on Privacy Enhancing Technologies*, 2019(1), 2019.
- [48] Thang Hoang, Attila Yavuz, F. Durak, and Jorge Guajardo. A multi-server oblivious dynamic searchable encryption framework. *Journal of Computer Security*, 27:1–28, 09 2019.
- [49] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. Batch codes and their applications. In *Proceedings of the Thirty-Sixth Annual ACM Symposium on Theory of Computing, STOC '04*, page 262–271, New York, NY, USA, 2004. Association for Computing Machinery.

- [50] Mohammad Saiful Islam, Mehmet Kuzu, and Murat Kantarcioglu. Access pattern disclosure on searchable encryption: Ramification, attack and mitigation. In *NDSS*, 2012.
- [51] Seny Kamara, Tarik Moataz, Andrew Park, and Lucy Qin. A decentralized and encrypted national gun registry. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 1520–1537, 2021.
- [52] Seny Kamara and Charalampos Papamanthou. Parallel and dynamic searchable symmetric encryption. In *International conference on financial cryptography and data security*, pages 258–274. Springer, 2013.
- [53] Seny Kamara, Charalampos Papamanthou, and Tom Roeder. Dynamic searchable symmetric encryption. In *Proceedings of the 2012 ACM conference on Computer and communications security*, pages 965–976, 2012.
- [54] Georgios Kellaris, George Kollios, Kobbi Nissim, and Adam O'Neill. Generic attacks on secure outsourced databases. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, CCS '16, page 1329–1340, New York, NY, USA, 2016. Association for Computing Machinery.
- [55] Shabnam Kasra Kermanshahi, Joseph K. Liu, Ron Steinfeld, Surya Nepal, Shangqi Lai, Randolph Loh, and Cong Zuo. Multi-client cloud-based symmetric searchable encryption. *IEEE Transactions on Dependable and Secure Computing*, 18(5):2419–2437, 2021.
- [56] Florian Kerschbaum and Anselme Tueno. An efficiently searchable encrypted data structure for range queries. In *European Symposium on Research in Computer Security*, 2017.
- [57] Aggelos Kiayias, Ozgur Oksuz, Alexander Russell, Qiang Tang, and Bing Wang. Efficient encrypted keyword search for multi-user data sharing. In *European symposium on research in computer security*, pages 173–195. Springer, 2016.
- [58] Evgenios M. Kornaropoulos, Charalampos Papamanthou, and Roberto Tamassia. The state of the uniform: Attacks on encrypted databases beyond the uniform query distribution. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 1223–1240, 2020.
- [59] Russell W. F. Lai and Sherman S. M. Chow. Forward-secure searchable encryption on labeled bipartite graphs. In *International Conference on Applied Cryptography and Network Security*, 2017.
- [60] Steven Lambregts, Huanhuan Chen, Jianting Ning, and Kaitai Liang. Val: Volume and access pattern leakage-abuse attack with leaked documents. In *European Symposium on Research in Computer Security*, pages 653–676. Springer, 2022.
- [61] Arthur Lazzaretti and Charalampos Papamanthou. Single pass Client-Preprocessing private information retrieval. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 5967–5984, Philadelphia, PA, August 2024. USENIX Association.
- [62] Tung Le, Rouzbeh Behnia, Jorge Guajardo, and Thang Hoang. MUSES: Efficient Multi-user Searchable Encrypted Database. In *33rd USENIX Security Symposium (USENIX Security 24)*, Philadelphia, PA, August 2024. USENIX Association.
- [63] Tung Le and Thang Hoang. MAPLE: A Metadata-Hiding Policy-Controllable Encrypted Search Platform with Minimal Trust. In *Proceedings on Privacy Enhancing Technologies Symposium (PETS)*, pages 184–203, 2023.
- [64] Tung Le and Thang Hoang. Hermes: Efficient and Secure Multi-Writer Encrypted Database. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 2865–2884, Los Alamitos, CA, USA, May 2025. IEEE Computer Society.
- [65] Tung Le and Thang Hoang. Efficient Single-Round Obfuscation of Search and Result Patterns in Searchable Encryption. Accepted in ACM Conference on Computer and Communications Security (CCS), 2026. <https://eprint.iacr.org/2026/1471>.
- [66] Hongbo Li, Qiong Huang, Jianye Huang, and Willy Susilo. Public-key authenticated encryption with keyword search supporting constant trapdoor generation and fast search. *IEEE Transactions on Information Forensics and Security*, 18:396–410, 2023.
- [67] Chang Liu, Liehuang Zhu, Mingzhong Wang, and Yu-An Tan. Search pattern leakage in searchable encryption: Attacks and new construction. *Inf. Sci.*, 265:176–188, may 2014.
- [68] Jacob R Lorch, Bryan Parno, James Mickens, Mariana Raykova, and Joshua Schiffman. Shroud: Ensuring private access to {Large-Scale} data in the data center. In *11th USENIX Conference on File and Storage Technologies (FAST 13)*, pages 199–213, 2013.
- [69] Travis Mayberry, Erik-Oliver Blass, and Agnes Hui Chan. Efficient private file retrieval by combining oram and pir. *NDSS* 2013, 2013.
- [70] Pratyush Mishra, Rishabh Poddar, Jerry Chen, Alessandro Chiesa, and Raluca Ada Popa. Oblix: An efficient oblivious search index. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 279–296. IEEE, 2018.
- [71] Priyanka Mondal, Javad Ghareh Chamani, Ioannis Demertzis, and Dimitrios Papadopoulos. I/O-Efficient dynamic searchable encryption meets forward & backward privacy. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 2527–2544, Philadelphia, PA, August 2024. USENIX Association.
- [72] Muhammad Naveed. The fallacy of composition of oblivious ram and searchable encryption. *Cryptology ePrint Archive*, Paper 2015/668, 2015. <https://eprint.iacr.org/2015/668>.
- [73] Hao Nie, Wei Wang, Peng Xu, Xianglong Zhang, Laurence T. Yang, and Kaitai Liang. Query recovery from easy to hard: Jigsaw attack against SSE. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 2599–2616, Philadelphia, PA, August 2024. USENIX Association.
- [74] Simon Oya and Florian Kerschbaum. Hiding the access pattern is not enough: Exploiting search pattern leakage in searchable encryption. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, August 2021.
- [75] Simon Oya and Florian Kerschbaum. IHOP: Improved statistical query recovery against searchable symmetric encryption through quadratic optimization. In *31st USENIX Security Symposium (USENIX Security 22)*, Boston, MA, August 2022.
- [76] David Pouliot and Charles V. Wright. The shadow nemesis: Inference attacks on efficiently deployable, efficiently searchable encryption. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, CCS '16, New York, NY, USA, 2016.
- [77] Daniel Pöllman and Tianxin Tang. Differentially private access in encrypted search: Achieving privacy at a small cost? *Cryptology ePrint Archive*, Paper 2025/1636, 2025.
- [78] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *J. ACM*, 56(6), September 2009.
- [79] Zhiwei Shang, Simon Oya, Andreas Peter, and Florian Kerschbaum. Obfuscated access and search patterns in searchable encryption. In *28th Annual Network and Distributed System Security Symposium*, NDSS 2021, 2021.
- [80] Victor Shoup et al. NTL: A library for doing number theory, 2001.
- [81] Dawn Xiaodong Song, D. Wagner, and A. Perrig. Practical techniques for searches on encrypted data. In *Proceeding 2000 IEEE Symposium on Security and Privacy*, S&P 2000, pages 44–55, 2000.
- [82] Emil Stefanov, Marten Van Dijk, Elaine Shi, T.-H. Hubert Chan, Christopher Fletcher, Ling Ren, Xiangyao Yu, and Srinivas Devasdas. Path ORAM: An Extremely Simple Oblivious RAM Protocol. *J. ACM*, 65(4), apr 2018.
- [83] Emil Stefanov, Charalampos Papamanthou, and Elaine Shi. Practical dynamic searchable encryption with small leakage. *Cryptology ePrint Archive*, Paper 2013/832, 2013. <https://eprint.iacr.org/2013/832>.
- [84] Shi-Feng Sun, Ron Steinfeld, Shangqi Lai, Xingliang Yuan, Amin Sakzad, Joseph K Liu, Surya Nepal, and Dawu Gu. Practical non-interactive searchable encryption with forward and backward privacy. In *NDSS*, 2021.
- [85] Shi-Feng Sun, Xingliang Yuan, Joseph K Liu, Ron Steinfeld, Amin Sakzad, Viet Vo, and Surya Nepal. Practical backward-secure searchable encryption from symmetric puncturable encryption. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 763–780, 2018.
- [86] Afonso Tinoco, Sixiang Gao, and Elaine Shi. Enigmap: Signal should use oblivious algorithms for private contact discovery. *Cryptology ePrint Archive*, 2022.
- [87] Raphael Toledo, George Danezis, and Ian Goldberg. Lower-cost epsilon-private information retrieval. *Proceedings on Privacy Enhancing Technologies*, 2016, 04 2016.
- [88] Adithya Vadapalli, Ryan Henry, and Ian Goldberg. Duoram: A Bandwidth-Efficient distributed ORAM for 2- and 3-party computation. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 3907–3924, Anaheim, CA, August 2023. USENIX Association.
- [89] Sameer Wagh, Paul Cuff, and Prateek Mittal. Root oram: A tunable differentially private oblivious ram. 01 2016.
- [90] Jiafan Wang and Sherman S. M. Chow. Omnes pro uno: Practical Multi-Writer encrypted database. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 2371–2388, Boston, MA, August 2022. USENIX Association.
- [91] Mingyue Wang, Yinbin Miao, Yu Guo, Hejiao Huang, Cong Wang, and Xiaohua Jia. Aesm2 attribute-based encrypted search for multi-owner and multi-user distributed systems. *IEEE Transactions on Parallel and Distributed Systems*, 34(1):92–107, 2023.
- [92] Wei Wang, Dongli Liu, Peng Xu, Laurence Tianruo Yang, and Kaitai Liang. Keyword search shareable encryption for fast and secure data replication. *IEEE Transactions on Information Forensics and Security*, 18:5537–5552, 2023.
- [93] Yun Wang and Dimitrios Papadopoulos. Multi-user collusion-resistant searchable encryption with optimal search time. In *Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security*, ASIA CCS '21, page 252–264, New York, NY, USA, 2021. Association for Computing Machinery.
- [94] Lei Xu, Leqian Zheng, Chengzhi Xu, Xingliang Yuan, and Cong Wang. Leakage-abuse attacks against forward and backward private searchable symmetric encryption. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, CCS '23, New York, NY, USA, 2023.
- [95] Peng Xu, Qianhong Wu, Wei Wang, Willy Susilo, Josep Domingo-Ferrer, and Hai Jin. Generating searchable public-key ciphertexts with hidden structures for fast keyword search. *IEEE Transactions on Information Forensics and Security*, 10(9):1993–2006, 2015.
- [96] Xianglong Zhang, Wei Wang, Peng Xu, Laurence T. Yang, and Kaitai Liang. High recovery with fewer injections: Practical binary volumetric injection attacks against dynamic searchable encryption. In *32nd USENIX Security Symposium (USENIX Security 23)*, 2023.
- [97] Yupeng Zhang, Jonathan Katz, and Charalampos Papamanthou. All your queries are belong to us: The power of File-Injection attacks on searchable encryption. In *25th USENIX Security Symposium (USENIX Security 16)*, Austin, TX, August 2016.